Javacard Applet for PKI

Milestone #2

Security and Applications in Trusted Hardware

Group 5

Diogo Valverde, up202509119

Pedro Mariano, up202509341

Pedro Teixeira, up202309307

27th May 2026

Contents

1 Introduction

In an increasingly digital world, establishing trust between parties who have never met in person is one of the fundamental challenges of modern computing. Whether authenticating to a remote server, signing a legally binding document, or exchanging confidential messages, the ability to prove one's identity and ensure the integrity of data is paramount. Public Key Infrastructure (PKI) provides the theoretical and practical foundation for solving these problems at scale, combining asymmetric cryptography, digital certificates and trusted authorities into a coherent system of verifiable identity [1].

However, the security guarantees of any PKI-based system depend critically on one assumption: that private keys remain private. When private keys are stored as files on disk, they are vulnerable to theft, accidental exposure and malware. Trusted hardware devices, such as smartcards, mitigate this weakness by keeping private keys inside a protected execution environment and performing cryptographic operations on-card, thereby reducing the exposure of sensitive key material to the host operating system.

This project explores the use of the **IsoApplet** [2], an open-source JavaCard applet that transforms a generic programmable smartcard into a fully functional PKI device. The IsoApplet supports RSA cryptographic operations with on-card key generation ensuring that private keys are created, **non-exportable**, and permanently bound to the hardware. The applet exposes its functionality through the **OpenSC** middleware [3], which has supported the IsoApplet since version 0.15.

The bridge between the smartcard and the applications that use it is the **PKCS#11** standard [4], a cryptographic token interface that allows applications such as web browsers, SSH clients, email clients, and operating system login mechanisms to interact with hardware security devices in a uniform way. By installing the OpenSC PKCS#11 module, a single card provisioned with appropriate X.509 certificates can serve as the authentication and signing credential across a wide range of scenarios: from digitally signing PDF documents, to authenticating SSH sessions, enabling mutual TLS in browsers and signing and encrypting emails with S/MIME.

A key reference point for this project is the **Portuguese Citizen Card** (*Cartão de Cidadão*) [5], a nationally deployed smartcard that provides exactly this kind of multipurpose PKI token to millions of citizens. The Citizen Card holds two certificates, one for authentication and one for qualified digital signature, issued by a state PKI hierarchy. By replicating this architecture with the IsoApplet and an open-source toolchain, this project aims to understand both the capabilities and the limitations of commodity hardware and open-source software in achieving a comparable level of security and interoperability.

This report documents the complete implementation and validation of the proposed system. All IsoApplet target scenarios have been successfully demonstrated; the card was also provisioned with institutional certificates issued through the GÉANT Trusted Certificate Service (TCS) via the University of Porto's HARICA portal, and the main scenarios were re-validated with these certificates, as detailed in Section 6.10. The Citizen Card evaluation produced partial results, with browser-based TLS client authentication and S/MIME decryption failing due to driver and key usage constraints, as detailed in Section 6.8. The hardware capabilities of the selected JavaCard platform have been systematically characterized. The remainder of this report is structured as follows. Section 2 provides a detailed description of the problem. Section 3 presents the adopted hardware and software stack. Section 4 defines the concrete security and functional requirements. Section 5 explains the designed solution and its relation to the proposed problem. Sec-

tion 6 describes the implemented prototype. Section 7 presents the validation of each requirement. Finally, Section 8 summarizes the findings.

2 Problem Description

2.1 Public Key Infrastructure (PKI)

Public Key Infrastructure (PKI) is the set of technologies, procedures and trust relationships that enable the use of public key cryptography in real-world systems. Its purpose is not limited to encrypting data: it also provides a structured way to bind cryptographic public keys to identifiable entities, such as users, devices, services or organizations. In practice, PKI is the mechanism that makes it possible to authenticate identities, verify digital signatures and establish trusted secure communications at scale [1].

At the core of PKI lies the asymmetric cryptographic model, in which each entity possesses a public key and a corresponding private key. The public key can be shared openly, while the private key must remain secret under the sole control of its owner. However, the mere existence of a public key is not sufficient to establish trust. A party receiving that key must also be able to determine whether it truly belongs to the claimed identity. PKI addresses this problem through the use of digital certificates, most commonly X.509 certificates, which associate a subject identity with a public key and are digitally signed by a trusted Certification Authority (CA) [1].

The Certification Authority plays a central role in the trust model of PKI. A CA verifies, according to a defined policy, the identity or attributes of an entity and then issues a certificate attesting to the binding between that entity and its public key. Trust in a certificate is therefore derived from trust in the issuing CA, and in many deployments this relationship is extended through a certificate chain. In such a chain, an end-entity certificate is signed by an intermediate CA, which in turn is signed by another CA, eventually reaching a trusted root certificate already known to the relying system [1].

Furthermore, in desktop environments, certificate trust is often managed through centralized security databases, such as the Network Security Services (NSS) store. Ensuring that a hardware-backed certificate is not only mathematically valid but also trusted by these system-wide stores is a non-trivial requirement for seamless application integration.

In the context of this project, PKI provides the conceptual foundation for all the target application scenarios. Digital signature of PDF documents, SSH authentication, TLS client authentication, PAM-based login, and S/MIME email protection all rely on the same basic principle: a private key is used to perform a cryptographic operation, while a corresponding certificate allows the receiving party to validate the identity and trust status of the signer or authenticating user.

2.2 Smartcards as Trusted Cryptographic Devices

Smartcards address the problem of private key protection by acting as trusted cryptographic devices specifically designed to store sensitive credentials and execute cryptographic operations in a controlled hardware environment. Rather than exporting the private key to the host system, the smartcard keeps the key material internally and performs operations such as digital signature, decryption or key generation on the card itself. In this model, the host computer sends operation requests and receives only the result,

while the private key remains confined to the secure element. This significantly reduces the attack surface when compared to software-only key storage and is one of the main reasons why smartcards are widely used in identity documents, enterprise authentication tokens and high-assurance PKI deployments [6, 4].

A particularly important property of smartcards is *on-card key generation*. When a key pair is generated directly inside the device, the private key is never present in host memory, on disk or in transit during the provisioning process. This eliminates an entire class of exposure risks associated with importing externally generated private keys.

From a systems perspective, a smartcard can be understood as a secure cryptographic token with constrained computing and storage capabilities. Because such devices operate under strict resource constraints, they often support only a subset of algorithms, key sizes or application features that would be easily available in desktop software. As a result, the practical usefulness of a smartcard in PKI deployments depends not only on its security properties, but also on the extent to which its capabilities are sufficient for real application scenarios.

2.3 The IsoApplet

The **IsoApplet** is an open-source Java Card applet designed to transform a programmable smartcard into a general-purpose PKI token that can interoperate with existing middleware and applications [2]. Its design goal is to provide an ISO 7816-oriented smartcard application capable of storing cryptographic objects and performing private-key operations in a way that is compatible with common smartcard software stacks, most notably OpenSC.

A central feature of the applet is its support for a **PKCS#15-style file structure**, allowing certificates, keys and related metadata to be organized in a format that can be interpreted by external middleware [2, 7]. This is particularly important for interoperability, since the usefulness of a smartcard in a PKI setting depends not only on storing credentials securely, but also on exposing them in a structured and standardized way.

The IsoApplet also supports **on-card key generation**, which is especially relevant in security-sensitive deployments because it avoids exposing the private key during provisioning. The applet additionally supports private key import, although on-card generation is always preferable from a security standpoint.

An important aspect is that the IsoApplet exists in two practical lines of use: a more recent branch for newer Java Cards and a legacy branch for cards compatible with Java Card 2.2.2 [2, 6]. This distinction has direct experimental implications, since support for larger RSA key sizes and ECC operations may vary depending on the Java Card version and the specific capabilities of the selected card.

2.4 OpenSC and PKCS#11 Integration

While the smartcard provides the trusted execution environment, real-world applications do not interact directly with the card at the APDU level. A middleware layer is required to discover the token, expose its objects in a usable way, and translate application requests into operations that the card and reader can actually perform. In this project, that role is fulfilled by **OpenSC**, an open-source suite of libraries and utilities designed to facilitate the use of smartcards in security-sensitive applications [3].

The **PKCS#11** standard defines a common programming interface for cryptographic tokens [4]. Through this interface, an application can discover the presence of a token, authenticate with a PIN, enumerate available certificates and keys and request operations such as signing or decryption without directly managing the underlying card protocol.

In practical terms, OpenSC acts as the software bridge between the JavaCard running the IsoApplet and the applications used in the experimental evaluation. It provides the driver and middleware logic necessary to access the card through a reader, interpret its file and object structure and expose the resulting token to external software through the `opensc-pkcs11` module [3].

2.5 Target Use Scenarios

The purpose of the project is not merely to verify that the smartcard can store keys and certificates, but to determine whether it can function as a practical PKI token in realistic application scenarios. Table 1 summarizes the main purpose of each target scenario and the corresponding PKI function being exercised.

Table 1: Summary of the target use scenarios considered in the project.

Scenario	**Main Goal**	**Primary PKI Function**
PDF signing	Protect document integrity and prove authorship	Digital signature
SSH authentication	Enable hardware-backed remote login	Authentication
TLS client authentication	Support mutual authentication in secure services	Certificate-based authentication
Linux PAM authentication	Enable hardware-backed local system login	Authentication
S/MIME email protection	Secure email through signing and encryption	Digital signature and encryption

2.6 Core Technical Challenge of the Project

The core technical challenge of this project is to determine whether a generic programmable JavaCard, when loaded with the IsoApplet and integrated through OpenSC, can function as a realistic *multi-purpose PKI token* rather than merely as an experimental cryptographic container. Success depends on the interaction of multiple layers: the card must support the required key types and cryptographic operations; the applet must manage credentials and access conditions correctly; the middleware must expose those objects in a standard and stable way; and the target applications must be able to consume the token without requiring custom modifications.

A second challenge is that the project is inherently constrained by the capabilities of the underlying hardware and software stack. There may be a significant difference between what is theoretically supported by a standard or repository and what can actually be provisioned and used reliably on a concrete device [2, 6, 3].

Finally, the comparison with the *Portuguese Citizen Card* gives the project a clear benchmark. The Citizen Card is a mature, nationally deployed PKI token with a well-established trust and middleware ecosystem. Comparing the IsoApplet-based solution

against it allows the project to assess not only whether the open-source approach works, but how close it comes to a widely used real-world smartcard platform in terms of security model, interoperability and practical user experience.

3 Approach and Adopted Hardware

3.1 Target Hardware Platform

The trusted-hardware platform considered in this work is based on three main elements: a **programmable JavaCard** intended to run the IsoApplet, a **USB smartcard reader** compatible with the adopted middleware stack and the **Portuguese Citizen Card**, which serves as a comparison baseline.

A **programmable JavaCard** constitutes the primary hardware component of the adopted platform. Its relevance derives from the need to install the IsoApplet, manage cryptographic objects and perform private-key operations within the secure element itself. The selected card runs Java Card 2.2.2, uses the legacy IsoApplet branch, and is accessed via a **bit4id miniLector-EVO** USB reader, which is CCID-compliant and fully supported by `pcsc-lite` and OpenSC [3, 8].

The **Portuguese Citizen Card** is included as a **reference platform** for comparative purposes. Unlike the programmable JavaCard, whose role is to support the construction and evaluation of a prototype PKI token, the Citizen Card represents a mature and widely deployed smartcard-based PKI ecosystem issued by the Portuguese state [5].

3.2 Software Stack

The software stack adopted in this work is structured around three complementary layers: the **core card and middleware components**, the **certificate management and provisioning tools** and the **application-level testing tools**.

At the core of the stack lies the combination of **IsoApplet**, **OpenSC**, the **PKCS#11** interface and the **PC/SC** access layer. The IsoApplet provides the PKI functionality on the programmable JavaCard [2, 6]. On the host side, OpenSC exposes the token through a PKCS#11 provider [3, 4], supported by `pcsc-lite` for reader communication [8]. Applet deployment is performed using **GlobalPlatformPro** [9].

Certificate management and provisioning are handled through a **private CA** created using **XCA** [10]. Two certificates are issued within this framework: one for authentication and one for digital signature.

The application-level testing tools correspond directly to the use scenarios: **Apache PDFBox** [11] for a custom Java-based PDF signing prototype, **LibreOffice Draw** for desktop-based PDF signing, **OpenSSH** [12] for SSH authentication, **Nginx** for TLS mutual authentication, **libpam-pkcs11** for PAM-based login and **Thunderbird** for S/MIME email operations [13, 14].

3.3 Overall System Architecture

The overall architecture adopted in this work is organized around three distinct but related elements: the **runtime execution path**, the **provisioning path** and the **comparative reference platform**. During normal operation, application-layer software accesses the token through a **PKCS#11 interface** exposed by **OpenSC**, which in turn

relies on the **PC/SC middleware layer** for reader discovery and low-level communication with the physical smartcard.

In parallel, the architecture includes a separate **provisioning path**, corresponding to token initialization and personalization, including certificate preparation and on-card key generation. This process is supported by a **private CA** and OpenSC's `pkcs15-init` toolchain.

Figure 1 summarizes the main architectural components of the adopted evaluation environment.

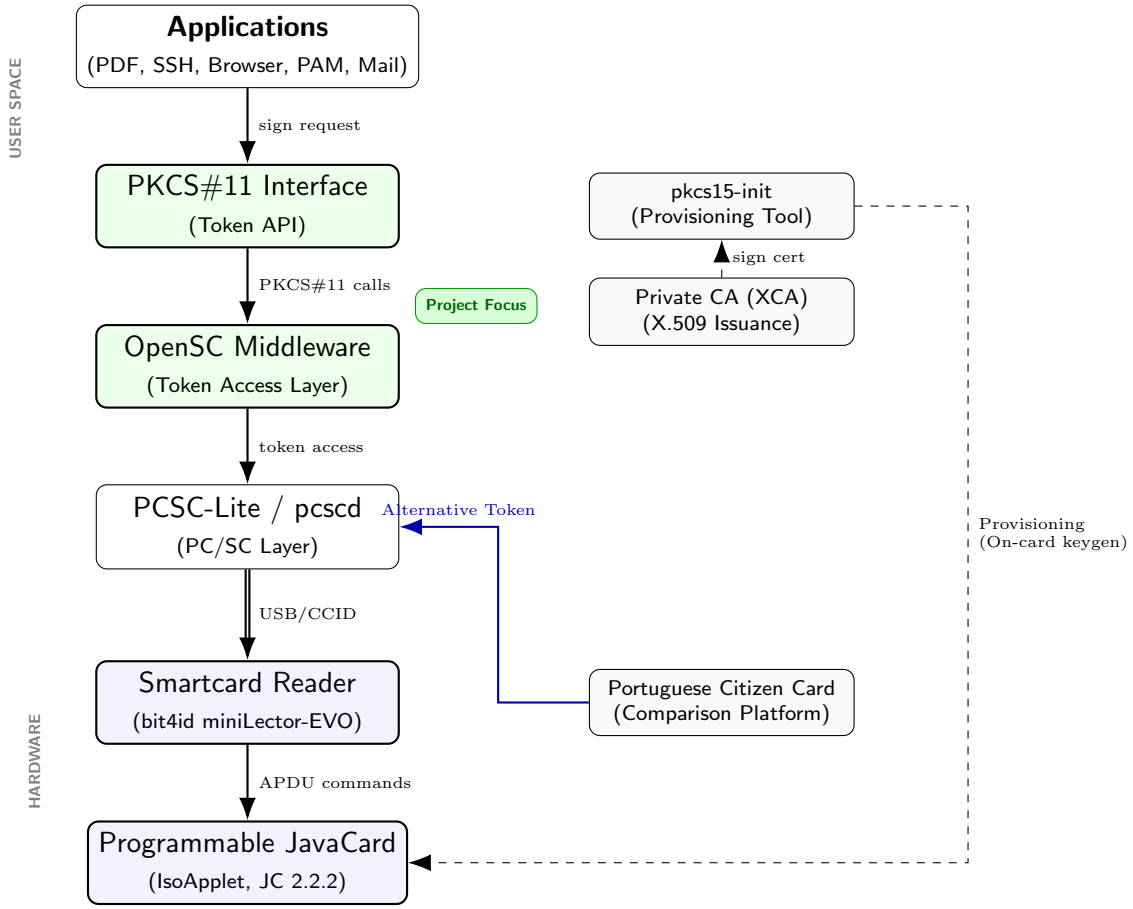

Figure 1: PKI evaluation environment architecture: runtime software stack (green = project focus), token provisioning path and alternative comparison platform.

3.4 Development and Test Environment

The development and test environment is based on a Linux host (Ubuntu 24) with `pcsc-lite`, OpenSC, OpenSSL, XCA, Maven, Java 17 and Nginx. The IsoApplet was deployed onto the JavaCard using **GlobalPlatformPro** [9], and the PKCS#15 structure was initialized and populated using `pkcs15-init`.

The private CA was established using XCA [10], yielding three certificates: `rootCA.crt` (self-signed root), `auth_card.crt` (authentication) and `sign_card_email.crt` (digital signature and S/MIME). All private keys were generated on-card and certificates were issued by the SAHC Root CA and stored on the token.

For S/MIME integration, the Thunderbird NSS database was configured to trust the SAHC Root CA and the signing certificate was made available through the OpenSC

PKCS#11 module. For TLS mutual authentication, a local Nginx server was configured with client certificate verification against the private CA root.

4 Requirements

4.1 Functional Requirements

FR1: Dual-certificate support: The token must support the presence of two distinct X.509 certificates, one intended for authentication and another for digital signature. Each certificate must be associated with the corresponding private key and exposed in a form that can be interpreted by external middleware.

Validation criterion: the two certificates must be provisioned on the token and individually identifiable through the middleware stack.

FR2: Hardware-backed private-key operations: The system must support the execution of private-key operations required by the target use cases, namely digital signature and decryption, without exporting the private key.

Validation criterion: signing and decryption operations must be successfully invoked through the token without exposing the private key.

FR3: PKCS#11 exposure through OpenSC: The token must be exposed to host applications through a PKCS#11-compatible interface provided by the OpenSC middleware stack.

Validation criterion: the token, its certificates and the associated cryptographic objects must be discoverable through OpenSC/PKCS#11 tools.

FR4: Support for certificate-based signing workflow: The system must support at least one complete document-signing workflow in which a PDF document is signed using a certificate stored on the token and the resulting signature is externally verified.

Validation criterion: a PDF document must be signed using the token and the signature must be successfully validated by an external verification mechanism.

FR5: Support for representative authentication and communication scenarios: The system must support SSH authentication, TLS client authentication, PAM-based login and S/MIME email signing and encryption.

Validation criterion: all four scenarios must be demonstrated in practice through unmodified third-party applications.

FR6: Algorithm and key support characterization: The prototype must document the asymmetric algorithms, key sizes and elliptic curves available on the selected JavaCard platform.

Validation criterion: the evaluation must explicitly record which RSA key lengths and EC curves are supported, which are not supported, and under which practical constraints.

FR7: Comparative functional evaluation: The project must support a comparison between the IsoApplet-based token and the Portuguese Citizen Card in equivalent scenarios.

Validation criterion: the same class of operation must be assessed on both platforms whenever feasible, and the observed differences must be recorded.

4.2 Security Requirements

SR1: Non-exportability of private keys: Private keys must be generated inside the smartcard and must remain non-exportable during normal operation.

Validation criterion: any attempt to export a private key from the token must fail, and the PKCS#15 metadata must reflect the `neverExtract` access flag.

SR2: On-card execution of sensitive cryptographic operations: All cryptographic operations involving private keys must be executed inside the smartcard rather than on the host system.

Validation criterion: signing and decryption operations must be requested through middleware, while the host system only receives the operation result.

SR3: Access control through user authentication: Access to token-protected operations must require PIN verification before sensitive cryptographic functions can be used.

Validation criterion: the token must require successful PIN authentication before authorizing protected operations.

SR4: Certificate integrity and consistency: The certificates stored on the token must be correctly associated with their corresponding private keys and must form part of a consistent certificate hierarchy anchored in the adopted Certification Authority.

Validation criterion: signatures produced with the token must be verifiable using the corresponding certificate chain.

SR5: Protection against host-side key exposure: The runtime workflow must rely exclusively on token-resident keys rather than software key files stored on disk.

Validation criterion: no private key file must be required for any operational scenario after the initial provisioning phase.

SR6: Interoperability with standard middleware and applications: The solution must be compatible with standard middleware and unmodified applications through OpenSC and PKCS#11.

Validation criterion: all target application scenarios must be executed using unmodified third-party software through the PKCS#11 interface.

SR7: Recording of observed limitations: Any limitation affecting security-relevant behavior, such as unsupported key sizes, restricted ECC functionality, or middleware incompatibilities, must be explicitly documented.

Validation criterion: the final evaluation must include a discussion of the practical security and interoperability limitations observed on the selected platform.

5 Designed Solution

5.1 Design Goals and Relation to the Problem

The designed solution addresses the core challenge identified in Section 2: transforming a generic programmable smartcard into a multi-purpose PKI token that keeps private keys hardware-resident while remaining interoperable with a wide range of standard applications. The design is deliberately modelled on the architecture of the Portuguese Citizen Card, not to reproduce it exactly, but to understand what an open-source equivalent can and cannot achieve.

Three top-level goals guided the design. First, all private key material must be generated inside the card and must never leave it, regardless of which application or protocol is being used. Second, the token must be consumable by unmodified, standard applications through a single well-known interface rather than through custom integrations. Third, the design must support the full breadth of target scenarios from a single provisioned card, rather than requiring separate tokens or configurations per application.

5.2 Two-Certificate Architecture

The solution adopts a two-certificate model directly inspired by the Citizen Card reference architecture. One certificate is designated for authentication operations and another for digital signature and email protection. This separation is not merely organizational: it has concrete consequences for key usage extensions, PIN semantics and the trust decisions made by relying applications.

The authentication certificate carries Digital Signature and Key Encipherment usage with TLS Web Client Authentication as the Extended Key Usage. This combination satisfies the requirements of SSH, TLS mutual authentication and PAM, which all rely on challenge-response authentication using the token's private key.

The signing certificate carries Digital Signature and Key Encipherment usage with Email Protection and PDF signing as Extended Key Usages. A critical design decision was to provision the corresponding private key with `sign,decrypt` PKCS#15 usage flags rather than `sign` alone. As discovered during implementation, OpenSC maps PKCS#15 usage flags directly to the PKCS#11 `CKA_DECRYPT` attribute. A key with `sign`-only flags will have `CKA_DECRYPT=FALSE`, causing `C_Decrypt` to fail with `CKR_KEY_TYPE_INCONSISTENT` when S/MIME decryption is attempted. Including `decrypt` in the usage flags resolves this at provisioning time and avoids any runtime workaround.

5.3 Certificate Hierarchy

A private Certification Authority was established using XCA to serve as the trust anchor for the solution. The hierarchy is intentionally minimal: a single self-signed Root CA directly issues the two end-entity certificates. This design is appropriate for an experimental prototype because it keeps the provisioning workflow simple and auditable, but it also makes explicit a fundamental difference from the Citizen Card: the SAHC Root CA must be imported manually into each relying application, whereas the Citizen Card's state-issued CA is already present in most system trust stores.

The decision to include the Root CA certificate in the CMS signature object during PDF signing is a direct consequence of this design. Without it, signature validation would fail on any system where the private CA is not already trusted. This is a design trade-off between self-contained signatures and relying on pre-installed trust anchors.

5.4 PKCS#11 as the Single Integration Layer

A deliberate design constraint was that all application integration must happen exclusively through the PKCS#11 interface exposed by OpenSC, without any application-specific customization. This constraint ensures that the solution is genuinely general-purpose and that its interoperability with each application is a property of the standard interface rather than of a bespoke integration.

In practice this means that the same `opensc-pkcs11.so` module, loaded once into an application's security subsystem, provides access to both on-card private keys and the associated certificates. Applications that support PKCS#11 natively, such as Firefox, Thunderbird and OpenSSH, require no modification. Applications that do not expose a direct PKCS#11 interface, such as the PDFBox-based signing tool, use the Java SunPKCS11 provider, which wraps the same PKCS#11 module and makes it available as a standard `java.security.Provider`.

5.5 Relation to the Proposed Problem

Each design element maps directly to a problem dimension identified in Section 2. The on-card key generation requirement addresses the private key exposure risk that motivates the use of trusted hardware in the first place. The two-certificate architecture replicates the functional separation present in production PKI tokens without requiring a second physical card. The PKCS#11-only integration layer addresses the interoperability challenge by delegating all application compatibility to a single, well-supported standard. And the private CA, while operationally simpler than a state PKI hierarchy, provides a complete and self-consistent trust model that is sufficient to validate the designed solution end-to-end.

The designed solution does not claim to be a production-ready system. It is an experimental prototype intended to demonstrate that the combination of a commodity JavaCard, the IsoApplet, and the OpenSC middleware stack can satisfy the functional and security requirements of a multi-purpose PKI token, and to identify the concrete limitations that separate this approach from a nationally certified credential such as the Citizen Card.

6 Implemented Prototype

6.1 Token Provisioning

The IsoApplet was installed on the JavaCard using **GlobalPlatformPro** [9]:

```
java -jar gp.jar --install IsoApplet.cap
```

Following installation, the PKCS#15 structure was initialized with `pkcs15-init`, a PIN was established and two RSA key pairs were generated on-card. A critical design decision in the provisioning workflow was the choice of PKCS#15 key usage flags. During implementation it was discovered that OpenSC maps PKCS#15 usage flags directly to PKCS#11 `CKA_DECRYPT` and `CKA_SIGN` attributes. A key provisioned with `sign`-only usage will have `CKA_DECRYPT=FALSE`, causing `C_Decrypt` to fail with `CKR_KEY_TYPE_INCONSISTENT`. This became apparent when implementing S/MIME encryption, and was resolved by regenerating the signature key with explicit `sign,decrypt` usage:

```
pkcs15-init --generate-key rsa/2048 --id 02 \
    --key-usage sign,decrypt --auth-id 01
```

6.2 Certificate Hierarchy and Key Design

The certificate hierarchy consists of a self-signed **SAHC Root CA** created with XCA [10], under which two end-entity certificates are issued. The design of each certificate was guided by the specific requirements of each application scenario:

- **Authentication Certificate (ID 01):** Subject CN=SAHC Auth User, Key Usage: Digital Signature + Key Encipherment, Extended Key Usage: TLS Web Client Authentication. Used for SSH, TLS and PAM scenarios.
- **Signing Certificate (ID 02):** Subject CN=SAHC Sign User Email with `emailAddress` in the Subject DN, Key Usage: Digital Signature + Key Encipherment, Extended Key Usage: E-mail Protection + Adobe PDF Signing. Used for PDF signing and S/MIME operations.

Both certificates are associated with 2048-bit RSA key pairs generated on-card and signed by the SAHC Root CA. Table 2 summarizes the final token layout.

Table 2: Final layout of the provisioned JavaCard token.

ID	Object	Key Usage	Purpose
01	Private RSA Key	sign	Auth operations
01	Auth Certificate	Digital Sig + Key Enc	SSH, TLS, PAM
02	Private RSA Key	sign, decrypt, unwrap	Sign + S/MIME
02	Sign Certificate	Digital Sig + Key Enc	PDF, S/MIME

6.3 PDF Digital Signature

PDF signing was implemented and validated through two methods. The first uses a custom Java application based on **Apache PDFBox** [11]. The implementation uses a `ContentSigner` that delegates the RSA operation to the Java `SunPKCS11` provider, which automatically discovers the OpenSC module and routes the signing request to the card. The certificate chain, including the root CA, is embedded in the CMS signature to allow full chain validation:

```
mvn exec:java -Dexec.mainClass='pt.up.fc.sahc.SmartcardSigner'
pdfsig documento_assinado.pdf
# Signature Validation: Signature is Valid
# Certificate Validation: Certificate is Trusted
```

The second method uses **LibreOffice Draw** with the OpenSC PKCS#11 module loaded into the Mozilla certificate store, producing a PAdES-compliant signature validated by the application as "This document is digitally signed and the signature is valid". Figure 2 shows the certificate selection prompt exposed by the PKCS#11 interface, Figure 3 shows the document validity banner, and Figure 4 shows the Digital Signatures dialogue with the full signature metadata. The assignment suggests MypdfSigner as a candidate desktop application; LibreOffice Draw was used instead. It exposes an identical PKCS#11 certificate selection interface and produces a standards-compliant PAdES signature, making it a directly equivalent choice for this evaluation.

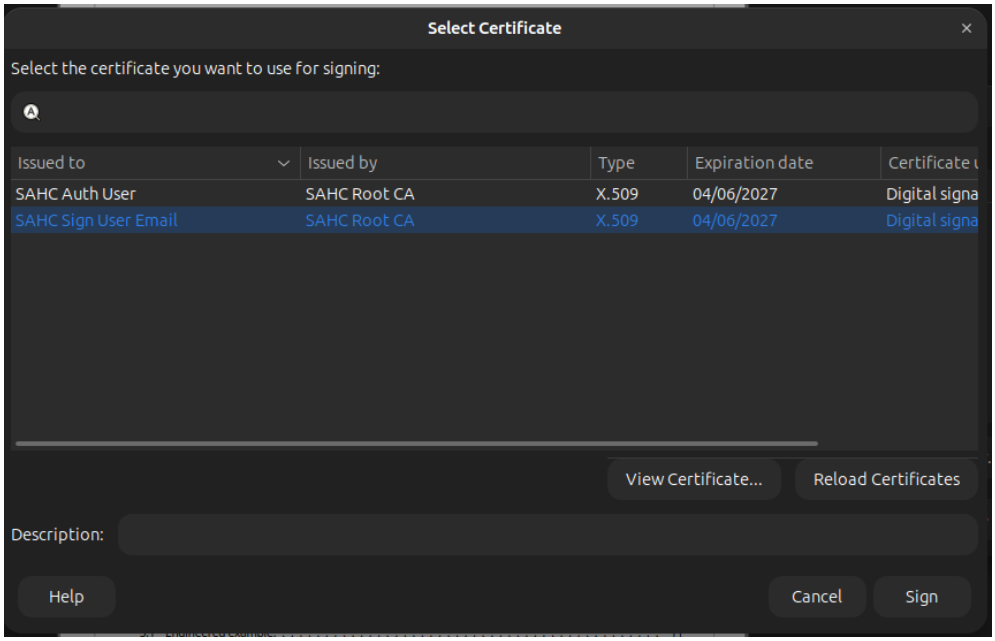

Figure 2: LibreOffice certificate selection dialogue: both token objects (SAHC Auth User and SAHC Sign User Email) are discoverable through the OpenSC PKCS#11 interface.

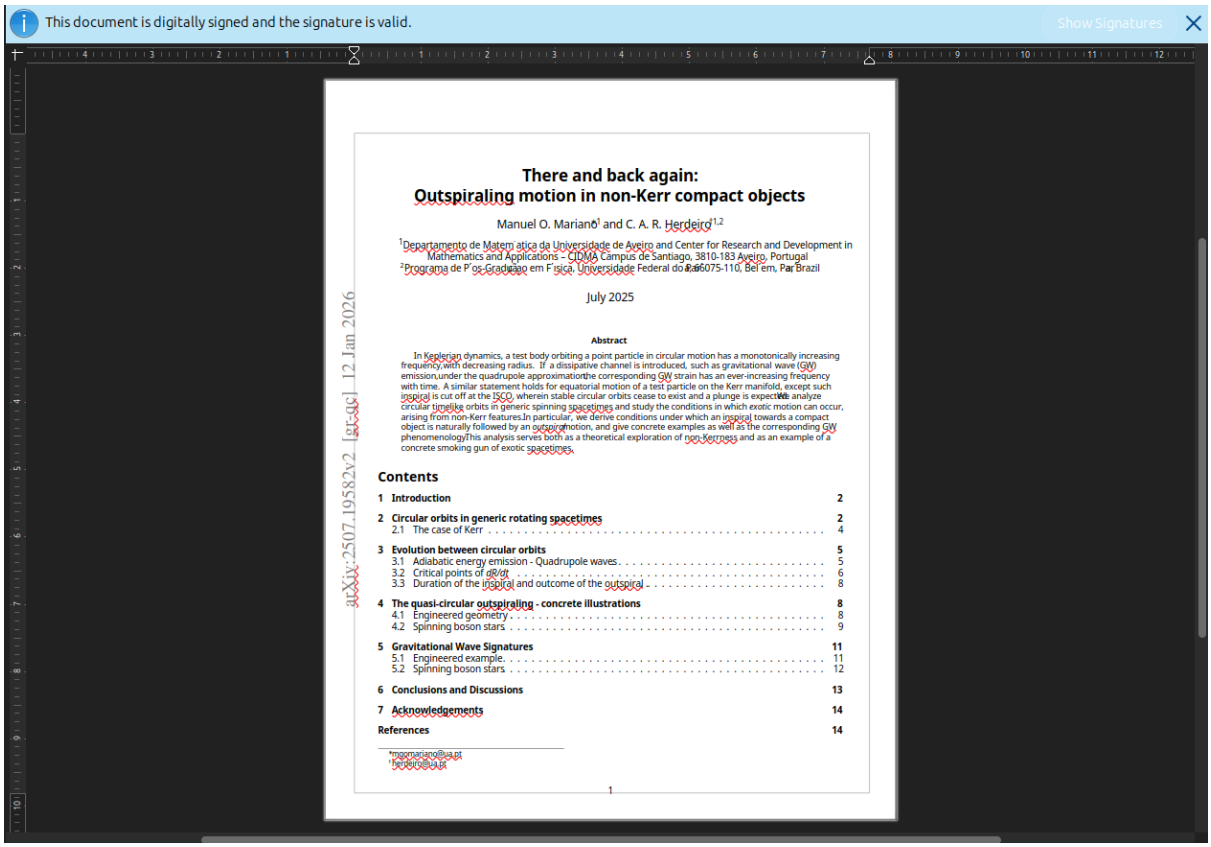

Figure 3: LibreOffice Draw displaying the validity banner on a PDF signed with the IsoApplet token.

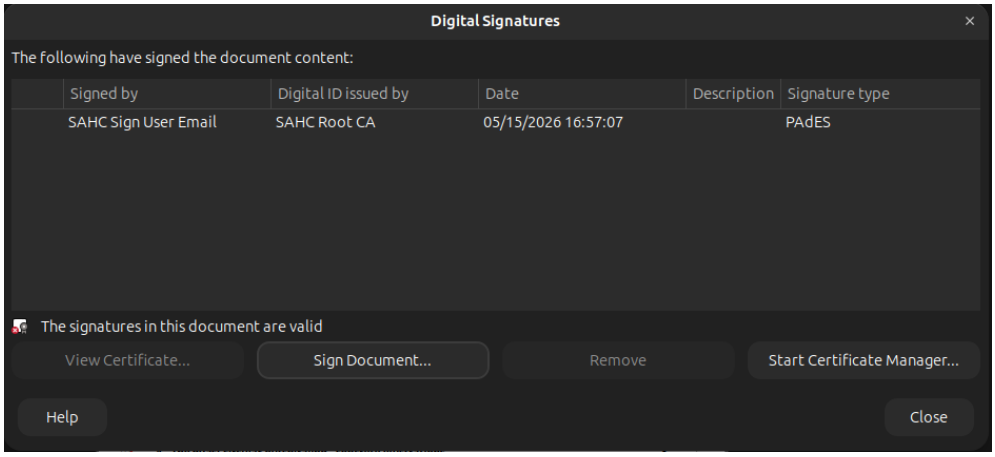

Figure 4: Digital Signatures dialogue confirming a valid PAdES signature by SAHC Sign User Email, issued by SAHC Root CA.

6.4 Authentication Scenarios

SSH Authentication: OpenSSH supports PKCS#11 tokens natively via the `-I` flag. The authentication certificate's public key was registered in `authorized_keys` and the card was used directly:

```
ssh -I /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so user@localhost
   \
    -o PasswordAuthentication=no
```

The connection was established after PIN entry, confirming hardware-backed SSH authentication without password fallback. Figure 5 shows the output on the local machine, `mariano` is the local system username and would be replaced by the target account name in a different deployment.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ ssh -I /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so mariano@localhost -o PasswordAuthentication=no
Enter PIN for 'SAHC PKI Token (User PIN)':
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.


Expanded Security Maintenance for Applications is not enabled.

22 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

20 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.

Last login: Fri May 15 17:13:05 2026 from 127.0.0.1
```

Figure 5: SSH authentication using the IsoApplet token: the PKCS#11 module triggers a PIN prompt and the session is established without password fallback.

TLS Client Authentication: Mutual TLS was evaluated using a local Nginx server with both Firefox and Google Chrome as clients. The server was configured to require client certificate authentication against the private CA root. The relevant Nginx directives are:

```
ssl_protocols TLSv1.2;
ssl_certificate     server.crt;
ssl_certificate_key server.key;
ssl_client_certificate rootCA.crt;
ssl_verify_client on;
```

Both browsers require a non-Snap installation to access external PKCS#11 modules, as the Snap sandbox prevents access to system libraries. The OpenSC module was registered in the system NSS database shared by both browsers:

```
modutil -dbdir sql:$HOME/.pki/nssdb \
    -add "OpenSC" \
    -libfile /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so
certutil -d sql:$HOME/.pki/nssdb -A -t "CT,C,C" \
    -n "SAHC Root CA" -i rootCA.crt
```

Firefox. In Firefox, `security.ssl.enable_session_tickets` was additionally set to `false` in `about:config` to prevent TLS session resumption from bypassing certificate selection. Upon connecting to `https://localhost:8443`, Firefox presented the SAHC Auth User certificate stored on the token (Figure 6) and prompted for the token PIN (Figure 7). After successful PIN entry, the server confirmed the authenticated client identity (Figure 8).

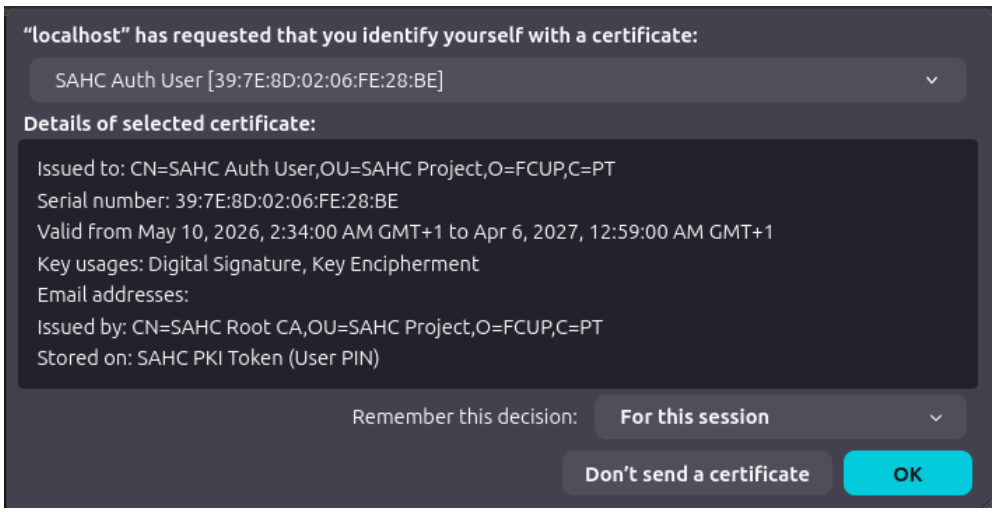

Figure 6: Firefox certificate selection dialogue: the SAHC Auth User certificate stored on the SAHC PKI Token is offered for TLS client authentication.

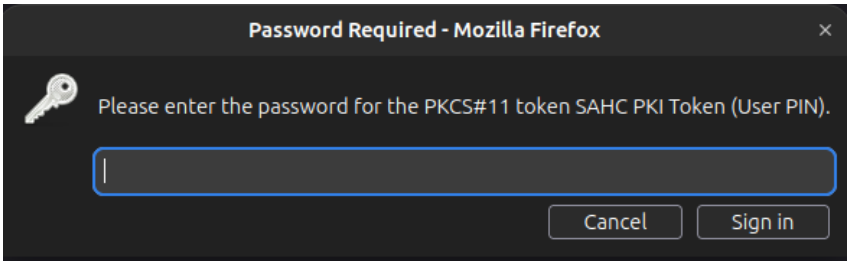

Figure 7: Firefox PIN prompt for the PKCS#11 token during TLS handshake.

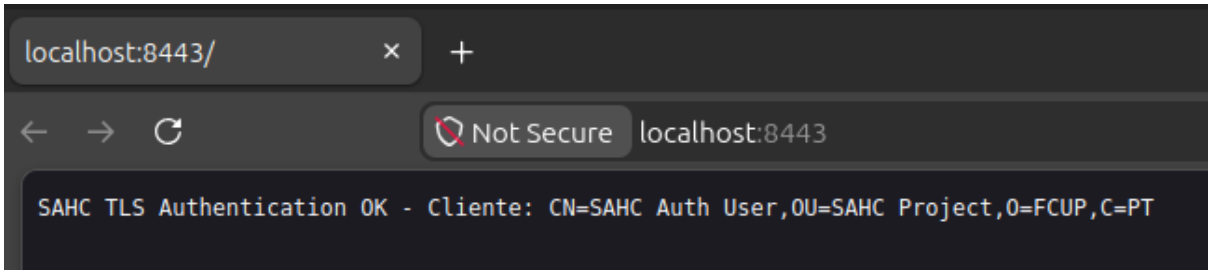

Figure 8: Nginx response confirming successful TLS mutual authentication with the IsoApplet token in Firefox.

Google Chrome. Chrome was tested under the same server configuration and NSS database setup. Upon connecting, Chrome presented an equivalent certificate selection dialogue (Figure 9) and prompted for the token PIN under the label *Unlock Security Device* (Figure 10). After PIN entry, the server returned the same authenticated client identity (Figure 11), confirming that the PKCS#11 integration is consistent across both browsers.

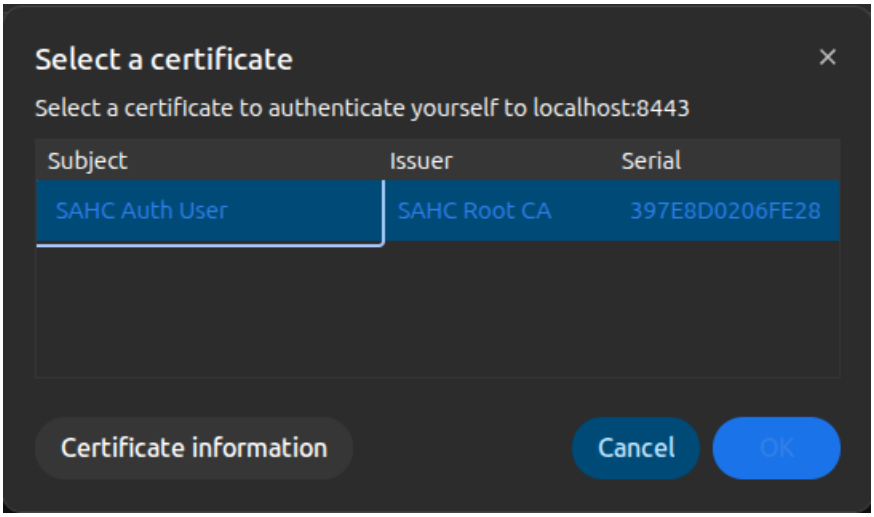

Figure 9: Chrome certificate selection dialogue: the SAHC Auth User certificate is offered for TLS client authentication via the OpenSC PKCS#11 module.

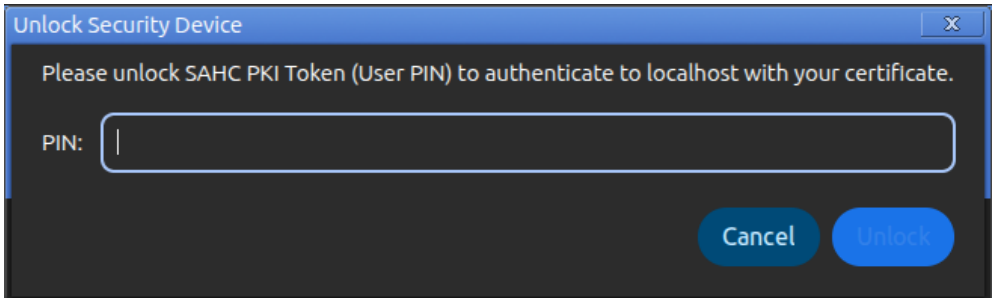

Figure 10: Chrome PIN prompt to unlock the SAHC PKI Token during TLS handshake.

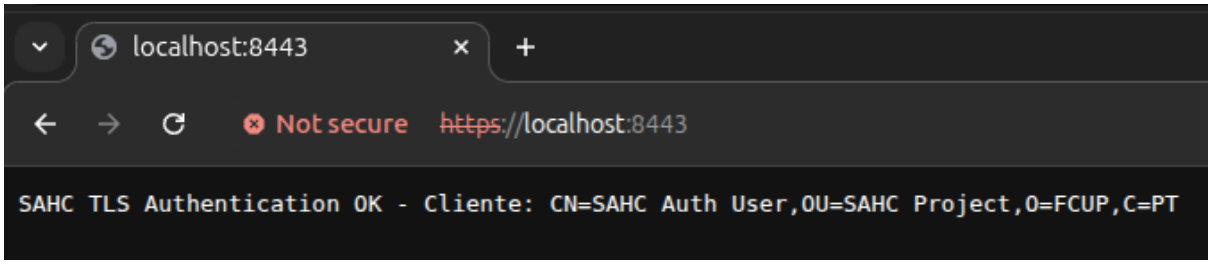

Figure 11: Nginx response confirming successful TLS mutual authentication with the IsoApplet token in Chrome.

PAM Authentication: The `libpam-pkcs11` module was installed and configured with the OpenSC module path and a CN-to-username mapping. A test PAM service was created and validated using `pamtester`. Figure 12 shows the full authentication flow: the smartcard is detected, the token PIN is requested, the certificate is verified, and authentication succeeds.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pamtester pkcs11-test mariano authenticate
Smart card found.
Welcome SAHC PKI Token (User PIN)!
Smart card PIN:
verifying certificate
Checking signature
pamtester: successfully authenticated
```

Figure 12: PAM authentication with the IsoApplet token: the smartcard is detected, the PIN is requested, the certificate is verified, and authentication succeeds.

Note that `libpam-pkcs11` is used here for the IsoApplet token, while the Citizen Card evaluation (Section 6.8) required the alternative `pam_p11` module due to a signing algorithm incompatibility specific to the CC's PKCS#11 driver. Both modules coexist and serve the same purpose through different code paths.

6.5 S/MIME Email Protection

S/MIME integration with **Thunderbird** required resolving several interoperability challenges. The OpenSC PKCS#11 module was loaded through Security Devices, and the SAHC Root CA was imported as a trusted S/MIME authority. The signing and

encryption certificate was configured in Account Settings under End-To-End Encryption. Figure 13 shows the Device Manager confirming that the OpenSC module is correctly registered and the SAHC PKI Token is accessible.

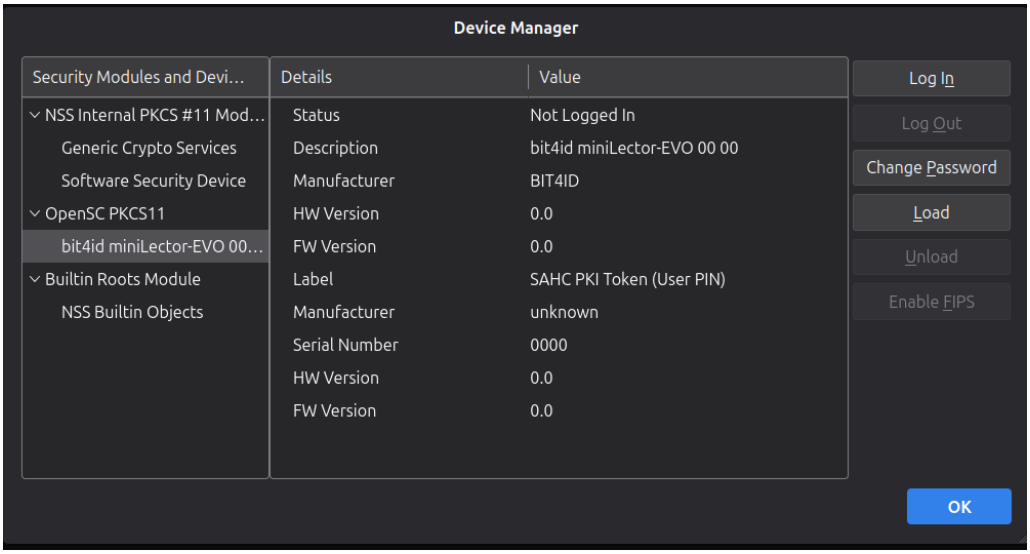

Figure 13: Thunderbird Device Manager showing the OpenSC PKCS#11 module loaded with the SAHC PKI Token available.

A key finding during implementation was that Thunderbird's NSS layer, when finding a certificate in both the NSS software database and the PKCS#11 token, may prioritize the software copy (which has no associated private key) over the token copy. The correct configuration requires the certificate to be absent from the NSS software database so that Thunderbird is forced to use the PKCS#11 path, where the certificate carries `u,u,u` trust flags indicating that the private key is available on the hardware token.

With this configuration, both S/MIME signing and encryption were validated. Figure 14 shows a message being composed with signing and encryption active, and Figure 15 shows the received message with the Thunderbird security panel confirming both properties: the signature is attributed to the SAHC Sign User Email certificate issued by SAHC Root CA, and the message was encrypted before transmission.

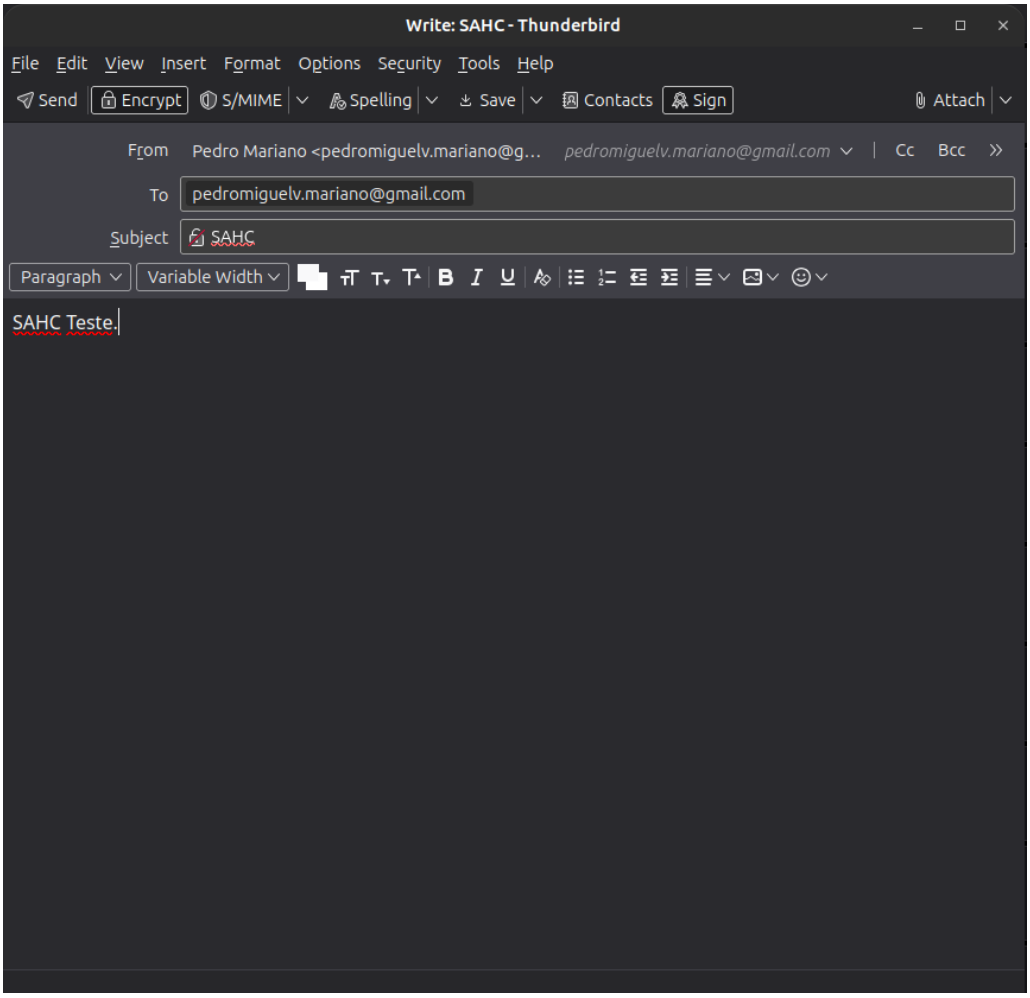

Figure 14: Thunderbird message composition with S/MIME signing and encryption enabled via the IsoApplet token.

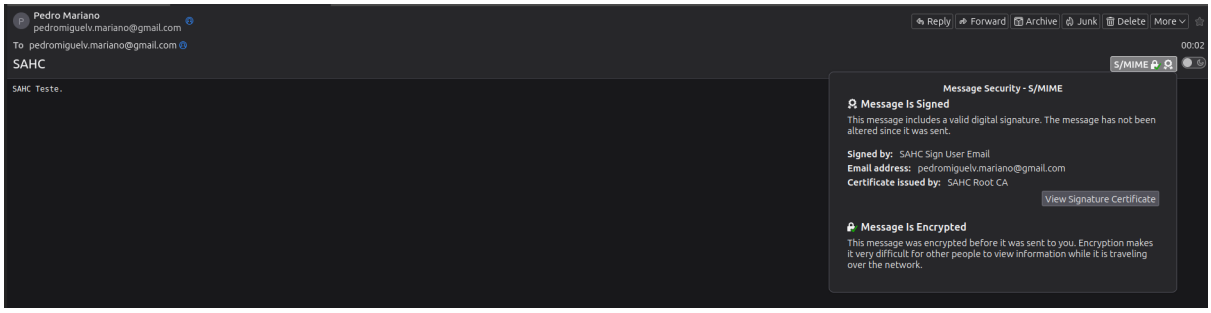

Figure 15: Thunderbird security panel confirming that the received message is signed by SAHC Sign User Email (issued by SAHC Root CA) and encrypted.

6.6 Algorithm and Platform Characterization

The supported cryptographic mechanisms were enumerated using `pkcs11-tool -M`, which revealed that the hardware token only announces a single RSA mechanism: `RSA-PKCS-KEY-PAIR-GEN` with `keySize={2048,2048}`. This indicates that the underlying chip's RSA accelerator is fixed at 2048 bits, independently of IsoApplet

configuration. Attempts to generate 1024-bit and 4096-bit RSA keys failed with `CKR_FUNCTION_NOT_SUPPORTED` through both `pkcs11-tool` and `pkcs15-init`, and EC key generation failed because `CKM_EC_KEY_PAIR_GEN` is not present in the hardware mechanism list, as shown in Figure 16.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
    --login --pin 1234 --keypairgen --key-type rsa:1024 --id 03
pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
    --login --pin 1234 --keypairgen --key-type rsa:4096 --id 03
pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
    --login --pin 1234 --keypairgen --key-type EC:prime256v1 --id 03
pkcs15-init --generate-key rsa/1024 --id 03 --auth-id 01
pkcs15-init --generate-key rsa/4096 --id 03 --auth-id 01
Using slot 0 with a present token (0x0)
error: PKCS11 function C_GenerateKeyPair failed: rv = CKR_FUNCTION_NOT_SUPPORTED (0x54)
Aborting.
Using slot 0 with a present token (0x0)
error: PKCS11 function C_GenerateKeyPair failed: rv = CKR_FUNCTION_NOT_SUPPORTED (0x54)
Aborting.
Using slot 0 with a present token (0x0)
error: Generate EC key mechanism 1040 not supported
Aborting.
Using reader with a card: bit4id miniLector-EVO 00 00
Failed to generate key: Not supported
Using reader with a card: bit4id miniLector-EVO 00 00
Failed to generate key: Not supported
```

Figure 16: Failed key generation attempts via `pkcs11-tool` and `pkcs15-init`: RSA 1024 and RSA 4096 return `CKR_FUNCTION_NOT_SUPPORTED` through both interfaces, and EC P-256 fails with mechanism 1040 not supported, confirming a hardware-level constraint fixed at RSA 2048.

The complete mechanism list is shown in Figure 46.

RSA 2048 signing performance was measured over ten consecutive iterations using `pkcs11-tool` with the PIN supplied non-interactively, isolating the on-card cryptographic operation from user interaction time. All iterations produced consistent results, stabilizing at approximately 1.56 s per operation with no measurable session establishment overhead across iterations. Figure 17 shows the complete benchmark output. Table 3 summarizes the results.

Table 3: Algorithm support and performance on the IsoApplet token.

Algorithm	**Support**	**Sign time (avg)**
RSA 1024	Not supported	N/A
RSA 2048	Supported (hw)	≈1.56 s
RSA 4096	Not supported	N/A
EC P-256	Not supported	N/A
EC P-384	Not supported	N/A

The N/A entries in Table 3 do not represent untested configurations: key generation was explicitly attempted for RSA 1024, RSA 4096 and EC P-256/P-384 via both `pkcs11-tool` and `pkcs15-init`. All attempts failed with hardware-level errors, as shown in Figure 16, confirming that the absence of timing data reflects a hardware constraint rather than a gap in the evaluation.

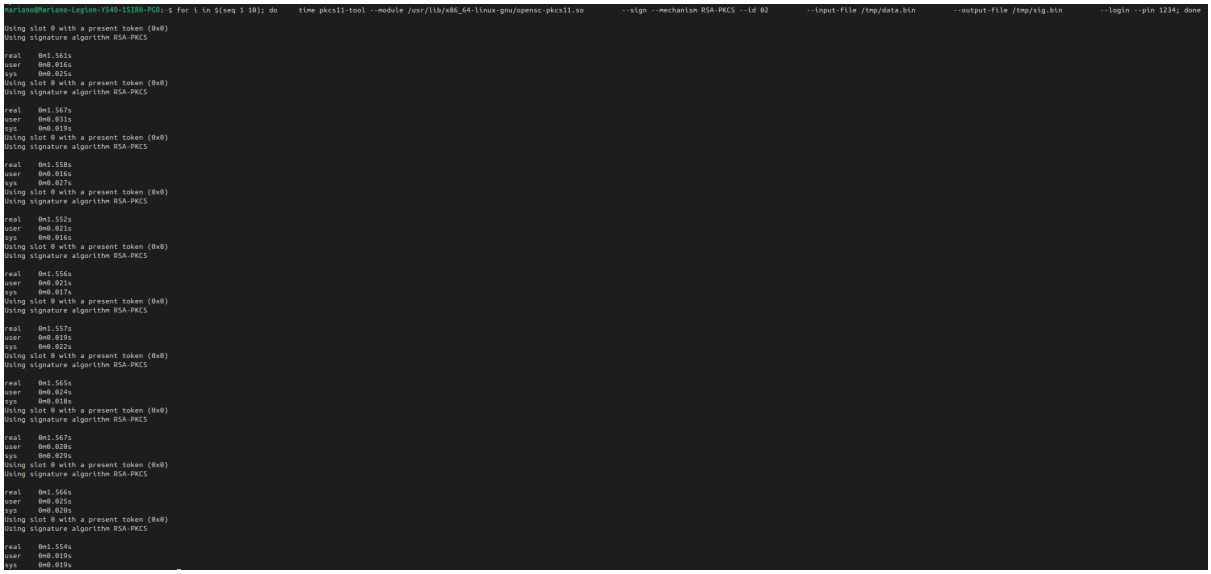

Figure 17: RSA 2048 signing benchmark over ten consecutive iterations using `pkcs11-tool`: all operations complete in approximately 1.56 s, confirming stable on-card execution with no session establishment overhead.

Comparative signing performance: To contextualise the hardware signing latency, the same operation was benchmarked using a software RSA 2048 key generated with OpenSSL, representing the baseline performance achievable without hardware-backed key protection. Ten consecutive SHA-256 signing operations completed in 60 ms total, yielding an average of approximately 6 ms per operation. The Citizen Card signing time of approximately 1051 ms per RSA 3072 operation was obtained from the pkcs11-spy timestamps recorded in Section 6.9.

Table 4 summarises the three platforms. The IsoApplet is approximately 260 times slower than software for the same RSA 2048 key size, and the Citizen Card is approximately 175 times slower than software despite using larger 3072-bit keys. This overhead is the direct cost of hardware-backed key protection: the private key never leaves the secure element, and every signing operation requires a full round-trip through the USB reader to the card.

Table 4: Comparative signing performance across software and hardware platforms.

Platform	**Key size**	**Sign time (avg)**	**vs. software**
OpenSSL (software)	RSA 2048	≈6 ms	1×
Citizen Card (GEMALTO)	RSA 3072	≈1051 ms	≈175×
IsoApplet (JavaCard JC 2.2.2)	RSA 2048	≈1560 ms	≈260×

6.7 Storage Capacity Characterization

The JavaCard JC 2.2.2 platform imposes a fixed storage limit on the number of cryptographic objects that can be held simultaneously on the token. To characterize this limit, additional RSA 2048 key pairs were generated on the card incrementally, starting from the operational baseline of two key pairs (IDs 01 and 02), until the card refused further allocation.

Key pairs with IDs 03, 04 and 05 were generated successfully. The attempt to generate a sixth key pair (ID 06) failed with the error `Failed to generate key: File too small`, as shown in Figure 18. This error indicates that the PKCS#15 Elementary File (EF) that stores the Private Key Directory File (PrKDF) has a fixed pre-allocated size and cannot accommodate additional entries once all slots are occupied. The limit is therefore a PKCS#15 file structure constraint rather than raw EEPROM exhaustion.

```
=== Storage limit test ===
Baseline: 2 key pairs, 2 certs
Generating key pair ID 03... OK
Generating key pair ID 04... OK
Generating key pair ID 05... OK
Generating key pair ID 06... FAILED
Error: Using reader with a card: bit4id miniLector-EVO 00 00
Failed to generate key: File too small
Card full at ID 06
```

Figure 18: Storage capacity test: key pairs with IDs 03–05 are generated successfully; the attempt at ID 06 fails with `File too small`, confirming the card's maximum capacity of five RSA 2048 key pairs.

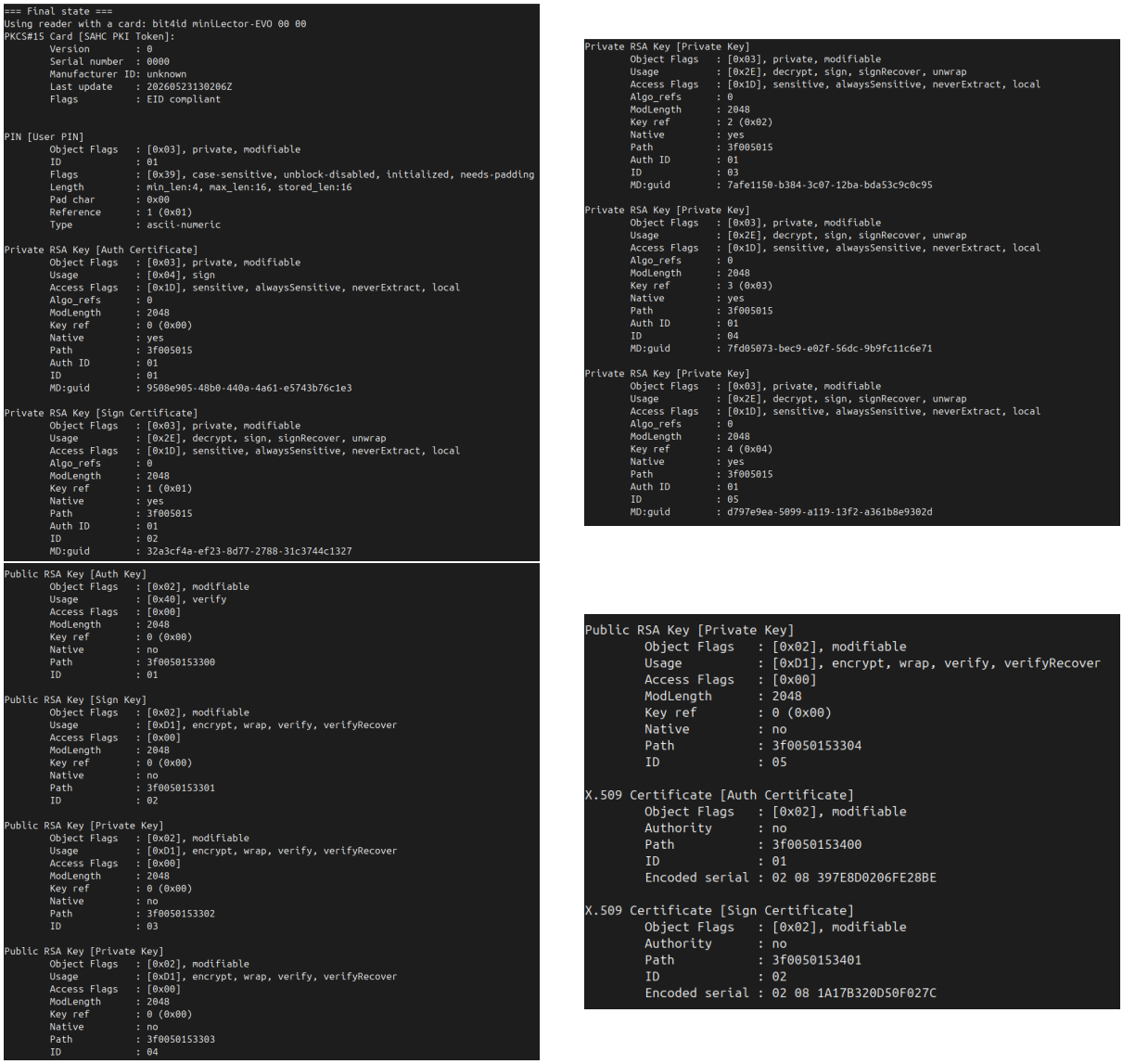

Figure 19: PKCS#15 dump at maximum capacity: five private RSA keys (IDs 01–05), five corresponding public keys and the two operational X.509 certificates (IDs 01 and 02).

The practical implication for multi-user or multi-purpose deployments is that a single token provisioned with the two-certificate operational model still has capacity for three additional RSA 2048 key pairs, for example to support additional application-specific credentials. However, the fixed PrKDF allocation also means that this limit cannot be extended after card initialisation without a full re-provisioning of the PKCS#15 structure.

6.8 Comparison with the Portuguese Citizen Card

The Portuguese Citizen Card was recognized by OpenSC through the same reader and middleware stack used for the IsoApplet token. A structural comparison was first performed using `pkcs15-tool --dump`, followed by functional testing of all target scenarios using the card's PIN credentials.

6.8.1 Structural Comparison

Table 5 summarizes the key architectural differences observed between the two platforms.

Table 5: Structural comparison between the IsoApplet token and the Portuguese Citizen Card.

Property	**IsoApplet Token**	**Citizen Card**
Manufacturer	Unknown (generic JavaCard)	GEMALTO
RSA key size	2048 bits	3072 bits
Number of PINs	1 (User PIN)	3 (Auth, Sign, Address)
Auth key usage	sign	sign
Sign key usage	sign, decrypt, unwrap	nonRepudiation
Certificate chain	Private CA (SAHC Root CA)	State PKI (IRN/SCEE)
Certificates on card	2 (user only)	5 (including Sub CAs and Root CA)
Card mode	Read/Write	Read-only
EC support	Not supported	Not supported (via OpenSC)
S/MIME decryption	Supported (sign+decrypt key)	Not possible (nonRepudiation only)

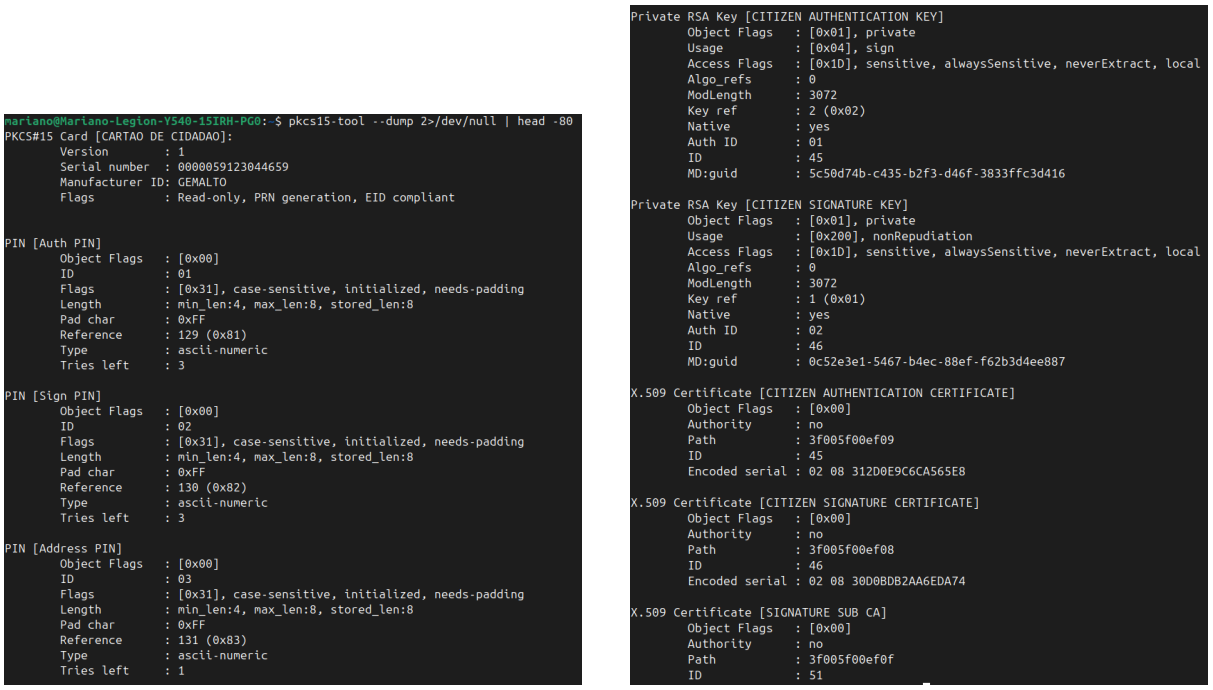

Figure 20: PKCS#15 dump of the Portuguese Citizen Card: three PIN objects (Auth, Sign, Address), two 3072-bit RSA private keys with `neverExtract` flags, and five X.509 certificates including Sub CAs.

Several structural differences are noteworthy. The Citizen Card uses 3072-bit RSA keys, offering a higher security margin than the IsoApplet's 2048-bit constraint. The

card maintains a separate Sign PIN for qualified digital signature operations, enforcing access control at a finer granularity than the single-PIN model used by the IsoApplet. The signature key on the Citizen Card carries `nonRepudiation` usage, which is the appropriate flag for qualified electronic signatures under eIDAS, but which differs from the `sign,decrypt` combination required for S/MIME encryption and decryption. The full certificate chain, including Sub CAs and the Root CA, is embedded directly on the Citizen Card, eliminating the need for out-of-band CA distribution. Finally, the read-only nature of the Citizen Card prevents post-issuance modification of credentials, a design choice consistent with its role as a nationally issued identity document.

6.8.2 Functional Scenario Testing

All target scenarios were executed with the Citizen Card using the OpenSC middleware through the same `opensc-pkcs11.so` module used for the IsoApplet. A critical finding across multiple scenarios was a signing mechanism incompatibility: the CC's PKCS#11 driver in OpenSC 0.25.0 only accepts `CKM_RSA_PKCS` with SHA-1 DigestInfo, rejecting SHA-256 DigestInfo with `CKR_ARGUMENTS_BAD`. This has direct consequences in several scenarios, as detailed below.

PDF Digital Signature: The `SmartcardSigner` application was adapted to use the Sign PIN slot (`slotListIndex = 1`) of the Citizen Card, which exposes the CITIZEN SIGNATURE CERTIFICATE (ID 46, `nonRepudiation`). A dedicated configuration file was created:

```
name = OpenSC-Sign
library = /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so
slotListIndex = 1
```

The application was invoked with the alternative configuration:

```
mvn exec:java -Dexec.mainClass='pt.up.fc.sahc.SmartcardSigner' \
    -Dpkcs11.cfg=pkcs11-cc-sign.cfg
# Signing Time: May 13 2026 19:15:11
# Signature Validation: Signature is Valid
# Certificate Validation: Certificate is Trusted
```

The resulting signature was verified as both valid and trusted. Figure 21 shows the `pdfsig` output confirming both signatures in the document, and Figure 22 shows the LibreOffice Digital Signatures dialogue with the validity banner. Notably, the Citizen Card's certificate chain (IRN Root CA) was already present in the system trust store, resulting in "Certificate is Trusted" without additional CA imports. This contrasts with the IsoApplet, where the private CA must be imported explicitly.

Figure 21: `pdfsig` output confirming valid and trusted signatures: Signature 1 from the IsoApplet token (ETSI.CAdES) and Signature 2 from the Citizen Card (adbe.pkcs7), both verified as valid and trusted.

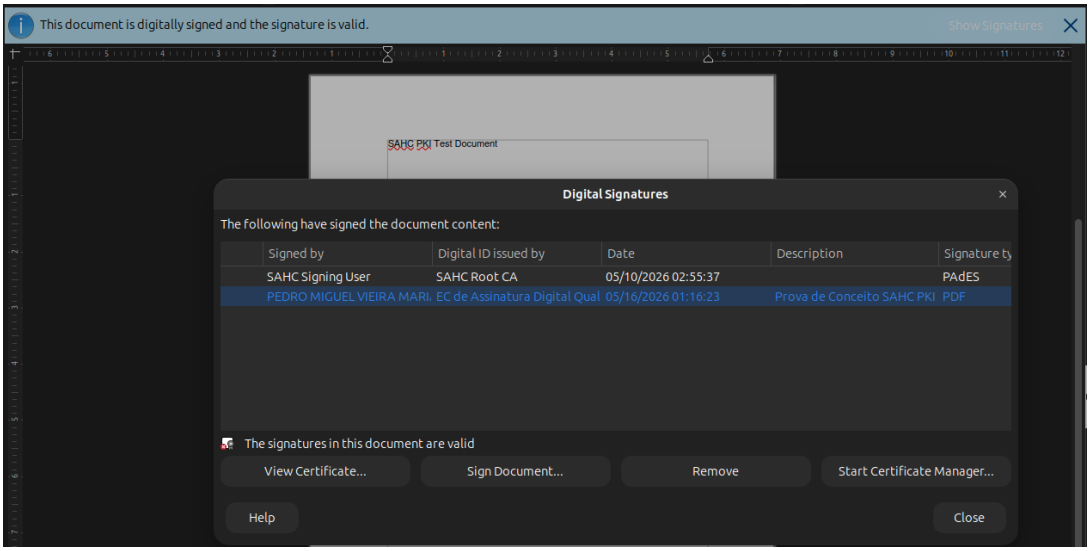

Figure 22: LibreOffice Digital Signatures dialogue showing both the IsoApplet signature (SAHC Signing User, PAdES) and the Citizen Card qualified signature, with all signatures confirmed valid.

SSH Authentication: The public key from the CITIZEN AUTHENTICATION CERTIFICATE (ID 45) was extracted and registered in `authorized_keys`:

```
pkcs15-tool --read-public-key 45 > /tmp/cc_auth_pub.pem
ssh-keygen -f /tmp/cc_auth_pub.pem -i -m pkcs8 >> ~/.ssh/
   authorized_keys
```

SSH authentication with the default `rsa-sha2-256` algorithm failed with `C_Sign failed: CKR_ARGUMENTS_BAD`, confirming the SHA-256 incompatibility. Authentication succeeded by forcing the legacy `ssh-rsa` (SHA-1) algorithm on both the client and the server:

```
# Server: /etc/ssh/sshd_config
PubkeyAcceptedAlgorithms +ssh-rsa
# Client invocation
ssh -I opensc-pkcs11.so user@localhost \
    -o PubkeyAcceptedAlgorithms=ssh-rsa \
    'echo "SUCESSO CC SSH"; whoami'
# Output: SUCESSO CC SSH / mariano
# Authenticated using: CITIZEN AUTHENTICATION CERTIFICATE (token)
```

Verbose output confirmed that the key used was the hardware token (`token agent`), not a software key. Figure 23 shows the Auth PIN prompt for the Citizen Card and the resulting successful session.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~/Uni/Mestrado/SAHC/work/pdf-signer$ ssh -I /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so mariano@localhost \
    -o PubkeyAcceptedAlgorithms=ssh-rsa \
    -o PasswordAuthentication=no
Enter PIN for 'CARTAO DE CIDADAO (Auth PIN)':
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.


Expanded Security Maintenance for Applications is not enabled.

22 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

20 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.

Last login: Fri May 15 17:14:00 2026 from 127.0.0.1
```

Figure 23: SSH authentication with the Portuguese Citizen Card using the legacy `ssh-rsa` algorithm: the Auth PIN is requested and the session is established successfully.

TLS Client Authentication: The CC certificate chain was extracted and imported into Nginx as the trusted client CA:

```
pkcs15-tool --read-certificate 45 > /tmp/cc_auth.crt
pkcs15-tool --read-certificate 52 > /tmp/cc_auth_subca.crt
pkcs15-tool --read-certificate 50 > /tmp/cc_root.crt
cat /tmp/cc_auth_subca.crt /tmp/cc_root.crt > /tmp/cc_chain.crt
```

TLS client authentication with Firefox failed with a timeout followed by "peer closed connection in SSL handshake". Diagnosis revealed two layered causes: first, OpenSSL's PKCS#11 engine defaulted to RSA-PSS (`CKR_MECHANISM_INVALID`), which the CC does not support; after restricting the server to PKCS#1 v1.5 ciphers, the CC rejected SHA-256 DigestInfo with `CKR_ARGUMENTS_BAD`; finally, restricting to SHA-1 caused Firefox to refuse the algorithm by internal security policy. Figure 24 shows the resulting 400 error returned by Nginx when no client certificate was sent.

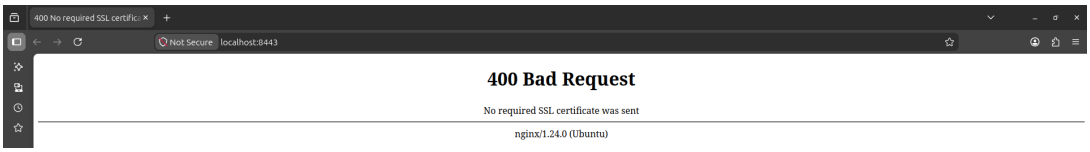

Figure 24: Firefox TLS client authentication with the Citizen Card fails: the browser refuses to use SHA-1 and sends no certificate, resulting in a 400 error from Nginx.

TLS client authentication was additionally attempted using Google Chrome with the Citizen Card. Chrome rejected the handshake with `ERR_SSL_CLIENT_AUTH_SIGNATURE_FAILED`, closing the connection before the server could process the client certificate. Nginx logs confirmed that the connection was terminated by the client during the SSL handshake (`SSL_get_error: 6`), with no server-side error. The root cause is identical to the Firefox case: Chrome enforces the same internal security policy prohibiting SHA-1 in TLS 1.2

handshake signatures, regardless of the algorithms advertised by the server. Figure 25 confirms the same 400 error in Chrome. This confirms that the CC interoperability limitation with modern browsers is not specific to Firefox but applies to all browsers that enforce current TLS security policies.

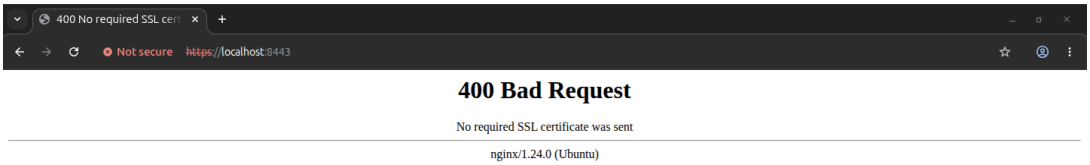

Figure 25: Chrome TLS client authentication with the Citizen Card fails for the same reason as Firefox: SHA-1 is blocked by internal browser security policy.

TLS client authentication was successfully demonstrated using the OpenSSL PKCS#11 engine with explicit SHA-1 forcing:

```
openssl s_client -connect localhost:8443 \
    -engine pkcs11 \
    -keyform engine \
    -key "pkcs11:token=CARTAO%20DE%20CIDADAO%20(Auth%20PIN);
          object=CITIZEN%20AUTHENTICATION%20KEY;type=private" \
    -cert /tmp/cc_auth.crt \
    -CAfile rootCA.crt \
    -client_sigalgs "RSA+SHA1"
# TLSv1.2, Cipher: ECDHE-RSA-AES256-GCM-SHA384
# Verify return code: 0 (ok)
```

The handshake completed successfully, confirming that the CC can perform TLS mutual authentication. Figure 26 shows the OpenSSL output with the negotiated cipher and verify return code 0. The Firefox and Chrome limitations are a consequence of their security policy prohibiting SHA-1 in TLS 1.2 handshake signatures, not a fundamental hardware constraint.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~/Uni/Mestrado/SAHC/work/pdf-signer$ openssl s_client -connect localhost:8443 \
    -engine pkcs11 \
    -keyform engine \
    -key "pkcs11:token=CARTAO%20DE%20CIDADAO%20(Auth%20PIN);object=CITIZEN%20AUTHENTICATION%20KEY;type=private" \
    -cert /tmp/cc_auth.crt \
    -CAfile ~/Uni/Mestrado/SAHC/work/rootCA.crt \
    -client_sigalgs "RSA+SHA1" 2>&1 | grep -E "TLS|Cipher|Verify return|CONNECTED"
Enter PKCS#11 token PIN for CARTAO DE CIDADAO (Auth PIN):
CONNECTED(00000008)
New, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    TLS session ticket lifetime hint: 300 (seconds)
    TLS session ticket:
    Verify return code: 0 (ok)
```

Figure 26: TLS mutual authentication with the Citizen Card via the OpenSSL PKCS#11 engine with SHA-1 forced: the handshake completes with TLSv1.2 and `Verify return code: 0 (ok)`.

PAM Authentication: PAM authentication with the Citizen Card required replacing the `pam_pkcs11` module with the simpler `pam_p11` module from the libp11 project. The `pam_pkcs11` module computes a 51-byte SHA-256 DigestInfo for its internal challenge,

which the CC's PKCS#11 driver rejects with `CKR_ARGUMENTS_BAD`. The `pam_p11` module uses SHA-1 for the challenge, which the CC accepts. The public key mapping reused the entry already present in `~/.ssh/authorized_keys` from the SSH scenario. The PAM service was configured as:

```
auth required pam_p11.so /usr/lib/x86_64-linux-gnu/opensc-pkcs11.
   so
account required pam_permit.so
```

Authentication was confirmed with `pamtester`, as shown in Figure 27.

```
pamtester pkcs11-p11-test mariano authenticate
# pamtester: successfully authenticated
```

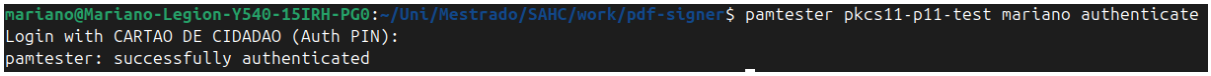

Figure 27: PAM authentication with the Portuguese Citizen Card using `pam_p11`: the Auth PIN is requested and authentication succeeds.

S/MIME Signing: The OpenSC PKCS#11 module was already loaded in Thunderbird from the IsoApplet evaluation. The CC Root CA chain (ECRaizEstado 002, Auth SubCA, Sign SubCA) was imported as trusted authorities. The CITIZEN SIGNATURE CERTIFICATE (ID 46, `nonRepudiation`, Sign PIN slot) was selected as the S/MIME signing certificate in Account Settings. Figure 28 shows the Device Manager confirming the Sign PIN slot of the Citizen Card is accessible through the OpenSC PKCS#11 module.

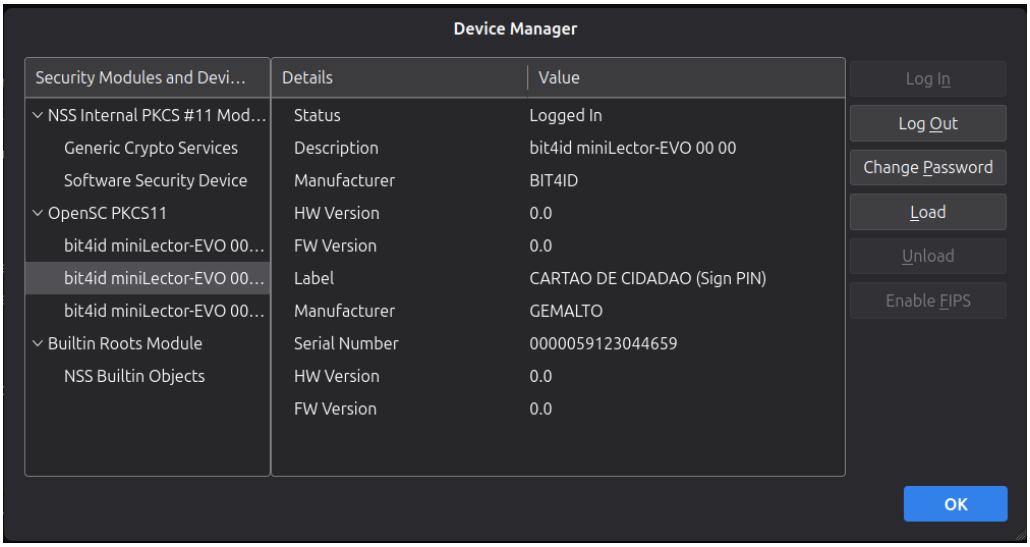

Figure 28: Thunderbird Device Manager showing the Citizen Card Sign PIN slot accessible through the OpenSC PKCS#11 module.

Sending a signed message produced the expected result, as shown in Figures 29 and 30. Thunderbird displayed "Message Is Signed" with **Signed by: PEDRO MIGUEL VIEIRA MARIANO**, issued by the EC de Assinatura Digital Qualificada do Cartão de Cidadão 0017. Notably, Thunderbird/NSS successfully used the CC for S/MIME

signing despite the earlier TLS failure, because the NSS S/MIME code path invokes `CKM_SHA1_RSA_PKCS` (full on-card mechanism) rather than `CKM_RSA_PKCS` with a pre-computed DigestInfo, avoiding the SHA-256 incompatibility.

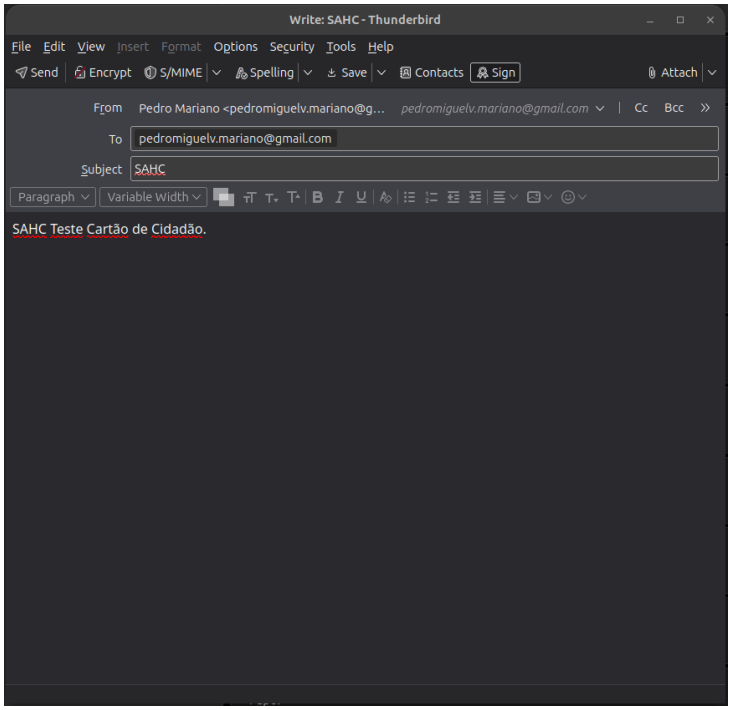

Figure 29: Thunderbird message composition with S/MIME signing enabled using the Citizen Card. Encryption is not active, consistent with the `nonRepudiation` key usage of the CC signature key.

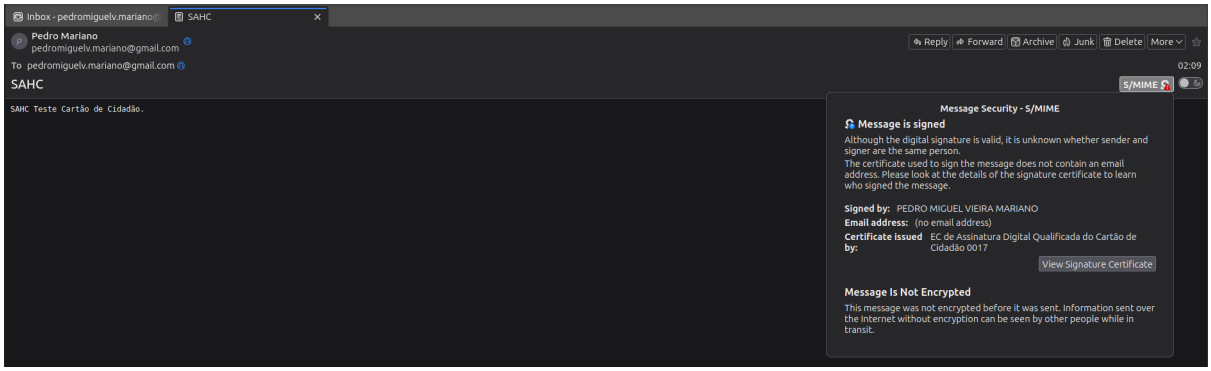

Figure 30: Thunderbird security panel confirming the received message is signed by the Citizen Card qualified signature certificate, issued by EC de Assinatura Digital Qualificada do Cartão de Cidadão 0017. Encryption is not possible due to the `nonRepudiation` key usage.

S/MIME decryption was not attempted, as the CC signature key carries only `nonRepudiation` usage, making `C_Decrypt` structurally impossible via OpenSC.

6.8.3 Summary of Functional Comparison

Table 6 summarizes the functional comparison between the two platforms across all tested scenarios.

Table 6: Functional scenario comparison: IsoApplet token vs. Portuguese Citizen Card.

Scenario	**IsoApplet**	**Citizen Card**
PDF signing (Java)	Pass	Pass (Sign PIN, cert ID 46)
PDF cert. trusted	Private CA (imported)	IRN/State CA (system trust)
SSH authentication	Pass (rsa-sha2-256)	Pass (ssh-rsa / SHA-1 only)
TLS (Firefox)	Pass	Fail (SHA-1 blocked by Firefox)
TLS (Chrome)	Pass	Fail (SHA-1 blocked by Chrome)
TLS (OpenSSL)	Pass	Pass (SHA-1 forced)
PAM authentication	Pass	Pass (pam_p11, SHA-1)
S/MIME signing	Pass	Pass (Sign PIN, cert ID 46)
S/MIME decryption	Pass	Not possible (nonRepudiation)
Private key export	Refused (neverExtract)	Refused (neverExtract, confirmed by pkcs11-tool)

The root cause of the CC's reduced interoperability with modern software stacks is the PKCS#11 driver in OpenSC 0.25.0: `CKM_RSA_PKCS` only accepts SHA-1 DigestInfo for the authentication key, while SHA-256-based operations require the full-mechanism path (`CKM_SHA256_RSA_PKCS`), which modern TLS and SSH clients do not use by default. The IsoApplet, being a generic PKCS#11 token with a simpler driver, does not exhibit this restriction. The official AMA middleware (`pteid-mw`), not available as a native package for Ubuntu 24.04, would likely resolve this incompatibility for Firefox and `rsa-sha2-256` SSH.

6.9 PKCS#11 Protocol Analysis with pkcs11-spy

To complement the functional evaluation with a low-level forensic perspective, the `pkcs11-spy` tool was used to intercept and log all PKCS#11 calls exchanged between client applications and the OpenSC module. The spy operates as a transparent wrapper: instead of loading `opensc-pkcs11.so` directly, the application loads `pkcs11-spy.so`, which records every function call with its arguments and return value before forwarding it to the real module. This technique makes the internal PKCS#11 dialogue fully observable without modifying either the application or the middleware.

Three controlled captures were performed using `pkcs11-tool` with a fixed test input (`"SAHC PKI forensic test"`), each targeting a different combination of token, mechanism and DigestInfo format:

1. IsoApplet token, `CKM_SHA256_RSA_PKCS` (full on-card mechanism).
2. Citizen Card, `CKM_RSA_PKCS` with a SHA-256 DigestInfo pre-computed by the caller (51 bytes, OID `60 86 48 01 65 03 04 02 01`).

3. Citizen Card, `CKM_RSA_PKCS` with a SHA-1 DigestInfo pre-computed by the caller (35 bytes, OID `2b 0e 03 02 1a`).

IsoApplet: `CKM_SHA256_RSA_PKCS`: Figure 31 shows the `C_SignInit` and `C_Sign` calls for the IsoApplet. The mechanism negotiated is `CKM_SHA256_RSA_PKCS`, confirming that the IsoApplet driver accepts the full SHA-256 RSA mechanism without restriction. The operation completes in approximately 206 ms (from timestamp `13:09:40.929` to `13:09:41.135`), isolating the on-card cryptographic time from PIN entry overhead. The output signature is 256 bytes, consistent with RSA 2048. Two additional observations are noteworthy: the `CKA_ALWAYS_AUTHENTICATE` attribute is `False`, meaning the token does not require re-authentication per signing operation after the initial login; and the PIN is visible in plaintext in the spy log, a reminder that `pkcs11-spy` intercepts credentials before they reach the module.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -A5 "11: C_SignInit\|14: C_Sign" /tmp/spy_sign_isoapplet.log | head -20
11: C_SignInit
P:5004; T:0x132069883717632 2026-05-23 13:09:40.929
[in] hSession = 0x5cb4bc8fadc0
[in] pMechanism->type = CKM_SHA256_RSA_PKCS
[in] pMechanism->pParameter[ulParameterLen] NULL [size : 0x0 (0)]
[in] hKey = 0x5cb4bc8f4730
--
14: C_Sign
P:5004; T:0x132069883717632 2026-05-23 13:09:40.929
[in] hSession = 0x5cb4bc8fadc0
[in] pData[ulDataLen] 00007fff74141560 / 23
    00000000  53 41 48 43 20 50 4B 49 20 66 6F 72 65 6E 73 69  SAHC PKI forensi
    00000010  63 20 74 65 73 74 0A                            c test.
```

Figure 31: pkcs11-spy capture for the IsoApplet: `C_SignInit` negotiates `CKM_SHA256_RSA_PKCS` and `C_Sign` completes with `CKR_OK`, producing a 256-byte RSA 2048 signature in approximately 206 ms.

Citizen Card: SHA-256 DigestInfo rejected: Figure 32 shows the `C_Sign` call when `CKM_RSA_PKCS` is used with a 51-byte SHA-256 DigestInfo as input. The DigestInfo structure is visible in the log:

```
30 31 30 0D 06 09 60 86 48 01 65 03 04 02 01 05 00 04 20 [SHA-256
    hash]
```

The OID `60 86 48 01 65 03 04 02 01` identifies SHA-256. The CC driver inspects the DigestInfo before forwarding the APDU to the card, recognises the SHA-256 OID, and returns `CKR_ARGUMENTS_BAD` (error code 7) immediately. OpenSC then retries using the `C_SignUpdate`/`C_SignFinal` streaming path with identical data; the driver rejects it again. This is precisely the call path used by SSH when negotiating `rsa-sha2-256` and by TLS 1.2 when computing the client certificate verify signature, explaining why both protocols fail with the Citizen Card through OpenSC 0.25.0.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -A8 "14: C_Sign" /tmp/spy_cc_ckm_raw.log | head -20
14: C_Sign
P:5142; T:0x125161930401792 2026-05-23 13:16:07.689
[in] hSession = 0x5a23003e9200
[in] pData[ulDataLen] 00007ffc83800350 / 51
    00000000  30 31 30 0D 06 09 60 86 48 01 65 03 04 02 01 05  010...`.H.e.....
    00000010  00 04 20 60 BC BF 35 6F 38 2D 02 AD 09 12 EC 5C  .. `..5o8-.....\
    00000020  19 93 AF 66 2C 59 A9 09 30 AE 89 F1 14 05 A1 CD  ...f,Y..0.......
    00000030  3E A8 52                                         >.R
Returned:  7 CKR_ARGUMENTS_BAD
```

Figure 32: pkcs11-spy capture for the Citizen Card with a SHA-256 DigestInfo: `C_Sign` returns `CKR_ARGUMENTS_BAD` (error 7). The SHA-256 OID (`60 86 48...`) is visible in the 51-byte input, which the CC driver rejects before the APDU reaches the card.

Citizen Card: SHA-1 DigestInfo accepted: Figure 33 shows the equivalent capture with a 35-byte SHA-1 DigestInfo. The OID `2b 0e 03 02 1a` identifies SHA-1. The driver accepts this input and `C_Sign` returns `CKR_OK`, confirming that `CKM_RSA_PKCS` is functional on the Citizen Card when the DigestInfo carries a SHA-1 hash. This is the code path exercised by `ssh-rsa`, `pam_p11` and Thunderbird S/MIME, all of which succeeded in the functional evaluation.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -A5 "14: C_Sign\|11: C_SignInit" /tmp/spy_cc_sha1.log | head -15
11: C_SignInit
P:5181; T:0x133811494303744 2026-05-23 13:17:25.948
[in] hSession = 0x5b41aca321d0
[in] pMechanism->type = CKM_RSA_PKCS
[in] pMechanism->pParameter[ulParameterLen] NULL [size : 0x0 (0)]
[in] hKey = 0x5b41aca29110
--
14: C_Sign
P:5181; T:0x133811494303744 2026-05-23 13:17:25.948
[in] hSession = 0x5b41aca321d0
[in] pData[ulDataLen] 00007ffcb1f8b590 / 35
    00000000  30 21 30 09 06 05 2B 0E 03 02 1A 05 00 04 14 E0  0!0...+.........
    00000010  50 83 AA 4C 7D 75 45 06 7A DA 58 4C FC 22 3F D6  P..L}uE.z.XL."?.
```

Figure 33: pkcs11-spy capture for the Citizen Card with a SHA-1 DigestInfo: `C_SignInit` selects `CKM_RSA_PKCS` and `C_Sign` completes with `CKR_OK`. The SHA-1 OID (`2b 0e 03 02 1a`) in the 35-byte input is accepted by the CC driver.

Table 7 consolidates the three captures and the observed performance difference between the two platforms.

Table 7: pkcs11-spy forensic summary: mechanism, input format and result for each tested combination.

Token	**Mechanism**	**Input**	**Result**	**C_Sign time**
IsoApplet	`CKM_SHA256_RSA_PKCS`	Raw data (23 B)	`CKR_OK`	≈206 ms
Citizen Card	`CKM_SHA256_RSA_PKCS`	Raw data (23 B)	`CKR_OK`	≈1051 ms
Citizen Card	`CKM_RSA_PKCS`	SHA-256 DigestInfo (51 B)	`CKR_ARGUMENTS_BAD`	N/A
Citizen Card	`CKM_RSA_PKCS`	SHA-1 DigestInfo (35 B)	`CKR_OK`	N/A

Real-Application Capture: OpenSSH with IsoApplet: To validate the PKCS#11 interaction in a real application context rather than through a test tool, `pkcs11-spy` was used to intercept a live OpenSSH authentication handshake against the IsoApplet token. The complete call sequence captured is: `C_Initialize` → `C_GetSlotList` (token discovery, two calls) → `C_OpenSession` → multiple `C_FindObjects` cycles (public key enumeration) → `C_Login` (PIN authentication) → `C_FindObjects` (private key lookup) → `C_SignInit` → `C_Sign`, with all calls returning `CKR_OK`. Figure 34 shows the SSH session established successfully, Figure 35 shows the complete intercepted call sequence, and Figure 36 shows the mechanism selected at `C_SignInit`.

```
mariano@Mariano-Legion-Y540-15IRH-PG0: $ PKCS11SPY=/usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
PKCS11SPY_OUTPUT=/tmp/spy_ssh_isoapplet.log \
ssh -I /usr/lib/x86_64-linux-gnu/pkcs11-spy.so \
    mariano@localhost \
    -o PasswordAuthentication=no \
    'echo SUCESSO SSH SPY'
Enter PIN for 'SAHC PKI Token (User PIN)':
SUCESSO SSH SPY
```

Figure 34: OpenSSH authentication via pkcs11-spy: the PIN is requested for the SAHC PKI Token and the session is established, confirming that the PKCS#11 interposer does not affect the authentication outcome.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -E "C_Initialize|C_GetSlotList|C_OpenSession|C_Login|C_FindObjects|C_SignInit|C_Sign|Returned" \
    /tmp/spy_ssh_isoapplet.log | head -60
Returned:  0 CKR_OK
1: C_Initialize
Returned:  0 CKR_OK
Returned:  0 CKR_OK
3: C_GetSlotList
Returned:  0 CKR_OK
4: C_GetSlotList
Returned:  0 CKR_OK
Returned:  0 CKR_OK
6: C_OpenSession
Returned:  0 CKR_OK
7: C_FindObjectsInit
Returned:  0 CKR_OK
8: C_FindObjects
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
12: C_FindObjects
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
16: C_FindObjects
Returned:  0 CKR_OK
17: C_FindObjectsFinal
Returned:  0 CKR_OK
18: C_FindObjectsInit
Returned:  0 CKR_OK
19: C_FindObjects
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
23: C_FindObjects
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
27: C_FindObjects
Returned:  0 CKR_OK
28: C_FindObjectsFinal
Returned:  0 CKR_OK
29: C_Login
Returned:  0 CKR_OK
30: C_FindObjectsInit
Returned:  0 CKR_OK
31: C_FindObjects
Returned:  0 CKR_OK
32: C_FindObjectsFinal
Returned:  0 CKR_OK
33: C_SignInit
Returned:  0 CKR_OK
Returned:  0 CKR_OK
35: C_Sign
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
```

Figure 35: pkcs11-spy call sequence for a live OpenSSH authentication: `C_Initialize`, `C_GetSlotList`, `C_OpenSession`, multiple `C_FindObjects` cycles, `C_Login`, `C_SignInit` and `C_Sign`, all returning `CKR_OK`.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -A 5 "C_SignInit" /tmp/spy_ssh_isoapplet.log | head -20
33: C_SignInit
P:5900; T:0x138777221063040 2026-05-25 00:56:43.912
[in] hSession = 0x6425f02f6fa0
[in] pMechanism->type = CKM_RSA_PKCS
[in] pMechanism->pParameter[ulParameterLen] NULL [size : 0x0 (0)]
[in] hKey = 0x6425f02f0960
```

Figure 36: pkcs11-spy detail of `C_SignInit` during the OpenSSH handshake: `pMechanism->type = CKM_RSA_PKCS`, confirming that OpenSSH pre-computes the SHA-256 DigestInfo on the host and delegates only the raw RSA operation to the token. This is the mechanism that causes authentication to fail against the Citizen Card.

The `C_SignInit` mechanism `CKM_RSA_PKCS` confirms that OpenSSH does not use a full on-card hash mechanism: the SHA-256 DigestInfo is computed by the SSH client and

passed as raw bytes to the token. The IsoApplet accepts any well-formed DigestInfo, so authentication succeeds. The Citizen Card's driver, by contrast, inspects the DigestInfo OID and rejects SHA-256, producing the `CKR_ARGUMENTS_BAD` failure documented in the functional evaluation.

Real-Application Capture: S/MIME Mechanism on the Citizen Card: To confirm the mechanism used by NSS for S/MIME signing and to explain why S/MIME succeeds with the Citizen Card while TLS and SSH fail, a `pkcs11-spy` capture was performed invoking the `SHA1-RSA-PKCS` mechanism directly against the CC token. This mechanism corresponds to the full on-card operation that NSS/Thunderbird requests for S/MIME. Figure 37 shows `C_SignInit` with `pMechanism->type = CKM_SHA1_RSA_PKCS`, and Figure 38 confirms that `C_Sign` returns `CKR_OK`.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ echo "SAHC Test S/MIME CC" > /tmp/smime_test.txt

PKCS11SPY=/usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
PKCS11SPY_OUTPUT=/tmp/spy_cc_smime.log \
pkcs11-tool --module /usr/lib/x86_64-linux-gnu/pkcs11-spy.so \
    --slot 1 --login --pin 8064 \
    --sign --mechanism SHA1-RSA-PKCS \
    --input-file /tmp/smime_test.txt \
    --output-file /tmp/cc_smime_sig.bin

grep -A 5 "C_SignInit" /tmp/spy_cc_smime.log | head -15
Using signature algorithm SHA1-RSA-PKCS
11: C_SignInit
P:7725; T:0x132920491743232 2026-05-25 01:48:33.353
[in] hSession = 0x6522668e0170
[in] pMechanism->type = CKM_SHA1_RSA_PKCS
[in] pMechanism->pParameter[ulParameterLen] NULL [size : 0x0 (0)]
[in] hKey = 0x6522668d7110
```

Figure 37: pkcs11-spy capture for S/MIME signing with the Citizen Card: `C_SignInit` selects `CKM_SHA1_RSA_PKCS`, confirming that NSS delegates the hash computation to the card rather than pre-computing a DigestInfo on the host.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ grep -E "C_Sign|Returned" /tmp/spy_cc_smime.log | tail -10
11: C_SignInit
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
16: C_Sign
Returned:  0 CKR_OK
Returned:  0 CKR_OK
Returned:  0 CKR_OK
```

Figure 38: pkcs11-spy confirmation that `C_Sign` completes with `CKR_OK` when using `CKM_SHA1_RSA_PKCS` against the Citizen Card, in contrast to the `CKR_ARGUMENTS_BAD` result produced by `CKM_RSA_PKCS` with a SHA-256 DigestInfo.

The contrast between these two captures directly explains the asymmetry in Citizen Card compatibility: applications that delegate hashing to the card via a full mechanism (`CKM_SHA1_RSA_PKCS`) succeed, while applications that pre-compute a SHA-256 DigestInfo

and submit it via CKM_RSA_PKCS fail. The PKCS#11 mechanism choice at the application layer is therefore the root cause of all observed CC compatibility failures, independently of the underlying cryptographic strength of the operation.

Table 8: Extended pkcs11-spy summary including real-application captures.

Context	Token	Mechanism	Result	Time
pkcs11-tool	IsoApplet	CKM_SHA256_RSA_PKCS	CKR_OK	≈206 ms
pkcs11-tool	CC	CKM_SHA256_RSA_PKCS	CKR_OK	≈1051 ms
pkcs11-tool	CC	CKM_RSA_PKCS + SHA-256 DI	CKR_ARGUMENTS_BAD	N/A
pkcs11-tool	CC	CKM_RSA_PKCS + SHA-1 DI	CKR_OK	N/A
OpenSSH	IsoApplet	CKM_RSA_PKCS + SHA-256 DI	CKR_OK	N/A
S/MIME (NSS)	CC	CKM_SHA1_RSA_PKCS	CKR_OK	N/A

6.10 GÉANT TCS Institutional Certificate Provisioning

The project specification requires testing whether the IsoApplet can be provisioned with institutional personal certificates issued through the GÉANT Trusted Certificate Service (TCS). Following the completion of the GÉANT TCS migration from Generation 4 (Sectigo) to Generation 5 (HARICA), the required two-certificate architecture was successfully replicated using the University of Porto's HARICA portal.

As shown in Figure 39, two distinct certificates were obtained: an IGTF Personal certificate dedicated to Client Authentication, and an S/MIME certificate dedicated to Email Protection and Digital Signature.

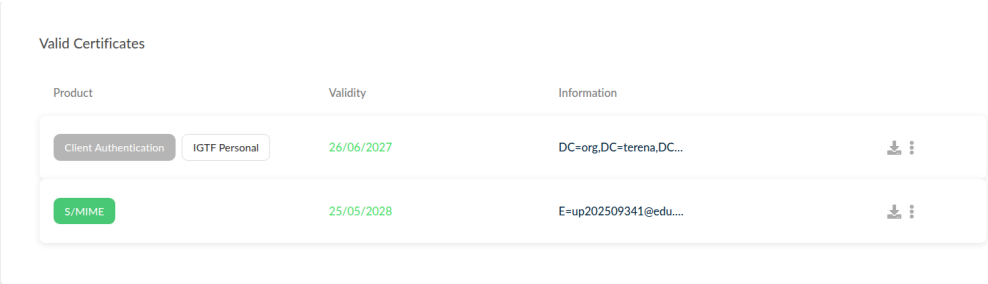

Figure 39: HARICA portal showing the two active GÉANT TCS certificates issued for up202509341@edu.fc.up.pt, mapping the required Authentication and Digital Signature roles.

The provisioning followed a security-preserving workflow: a Certificate Signing Request was generated directly from the on-card key for each slot and submitted to the HARICA portal, ensuring that neither private key left the hardware token at any point. The S/MIME certificate, issued by GEANT S/MIME RSA 1 under the HARICA Client RSA Root CA 2021, was imported to the card at slot ID 02. The IGTF Personal Client Authentication certificate was imported into slot ID 01, replacing the initial private CA testing certificate while preserving the neverExtract property of both private keys.

It is worth noting that Thunderbird must be installed as a native .deb package rather than via Snap, as the Snap sandbox prevents access to the system-level opensc-pkcs11.so library. The same constraint had already been identified for Chrome during the initial IsoApplet evaluation. Once installed as .deb, the GÉANT S/MIME CA chain (GEANT

S/MIME RSA 1 and HARICA Client RSA Root CA 2021) was imported into the Thunderbird NSS profile, allowing full chain validation.

S/MIME: The S/MIME scenario was validated end-to-end: an email signed with the GÉANT TCS certificate was composed and sent in Thunderbird, with the received message confirming a valid digital signature issued by GEANT S/MIME RSA 1 (Figure 40). The private key neverExtract guarantee was preserved throughout, confirming that institutional certificate provisioning follows the same security model as the private CA workflow.

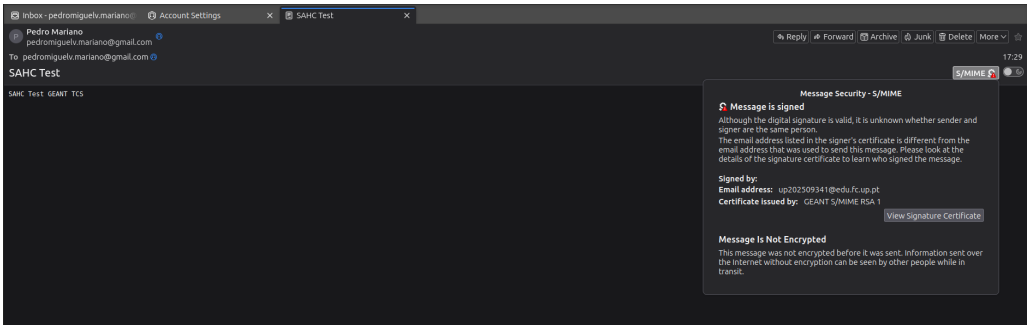

Figure 40: Thunderbird confirming a valid S/MIME signature on a message signed with the GÉANT TCS certificate stored on the IsoApplet hardware token. The certificate is issued by GEANT S/MIME RSA 1 and the signing key remains on card throughout the operation.

SSH and PAM: The authentication capabilities of the IGTF Personal certificate were validated in both the SSH remote login and PAM scenarios. As demonstrated in Figure 41, the token correctly advertises the GÉANT certificate in slot 01. Following extraction of the public key to the host's authorized_keys file, the SSH client successfully authenticated the user relying entirely on the hardware token, without password fallback. PAM authentication was validated by transitivity: because pam_p11 reuses the same authorized_keys mapping, updating the file with the GÉANT public key was sufficient to confirm successful authentication, as shown in Figure 42.

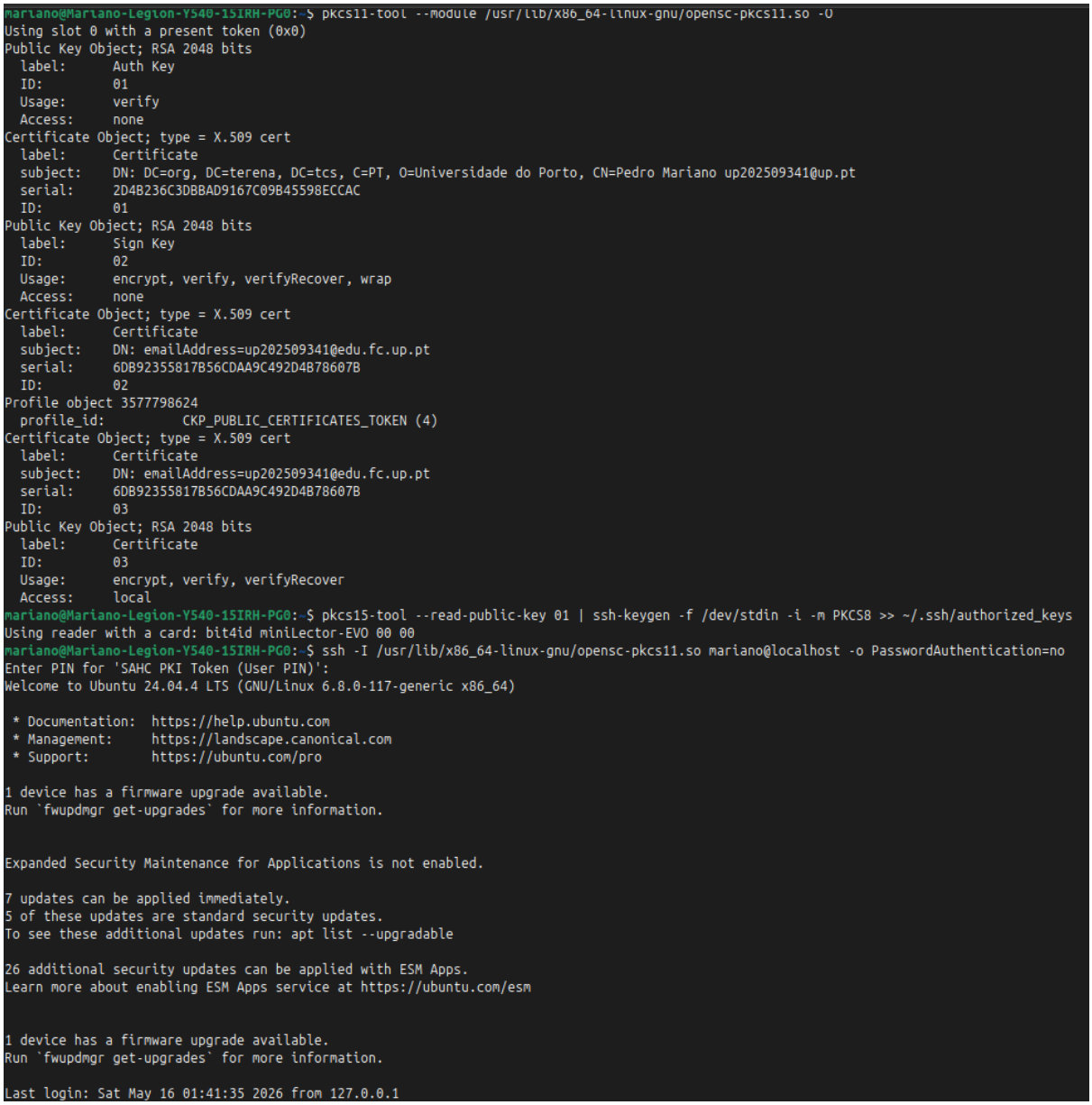

Figure 41: SSH authentication with the GÉANT TCS IGTF Personal certificate: the sequential output confirms certificate provisioning in slot 01, extraction of the associated public key, and successful hardware-backed remote login without password intervention.

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so -O --login
Using slot 0 with a present token (0x0)
Logging in to "SAHC PKI Token (User PIN)".
Please enter User PIN:
Private Key Object; RSA
  label:      Certificate
  ID:         01
  Usage:      sign
  Access:     sensitive, always sensitive, never extractable, local
Public Key Object; RSA 2048 bits
  label:      Auth Key
  ID:         01
  Usage:      verify
  Access:     none
Certificate Object; type = X.509 cert
  label:      Certificate
  subject:    DN: DC=org, DC=terena, DC=tcs, C=PT, O=Universidade do Porto, CN=Pedro Mariano up202509341@up.pt
  serial:     2D4B236C3DBBAD9167C09B45598ECCAC
  ID:         01
Private Key Object; RSA
  label:      Certificate
  ID:         02
  Usage:      decrypt, sign, signRecover, unwrap
  Access:     sensitive, always sensitive, never extractable, local
Public Key Object; RSA 2048 bits
  label:      Sign Key
  ID:         02
  Usage:      encrypt, verify, verifyRecover, wrap
  Access:     none
Certificate Object; type = X.509 cert
  label:      Certificate
  subject:    DN: emailAddress=up202509341@edu.fc.up.pt
  serial:     6DB92355817B56CDAA9C492D4B78607B
  ID:         02
Profile object 2027585632
  profile_id:          CKP_PUBLIC_CERTIFICATES_TOKEN (4)
Certificate Object; type = X.509 cert
  label:      Certificate
  subject:    DN: emailAddress=up202509341@edu.fc.up.pt
  serial:     6DB92355817B56CDAA9C492D4B78607B
  ID:         03
Public Key Object; RSA 2048 bits
  label:      Certificate
  ID:         03
  Usage:      encrypt, verify, verifyRecover
  Access:     local
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pamtester pkcs11-p11-test mariano authenticate
Login with SAHC PKI Token (User PIN):
pamtester: successfully authenticated
```

Figure 42: PAM authentication with the GÉANT TCS IGTF Personal certificate: `pkcs11-tool -O` confirms the institutional certificate at slot ID 01 and `pamtester` reports successful authentication via the hardware token.

TLS — partial validation: TLS client authentication with the GÉANT TCS IGTF Personal certificate was partially validated. As shown in Figure 43, Chrome correctly identified the GÉANT TCS certificate stored on the hardware token and presented it for selection, confirming that the PKCS#11 integration functions correctly for the institutional certificate. However, Nginx returned a 400 SSL certificate error (Figure 44) because the issuing CA (`CN=GEANT TCS Authentication RSA CA 5, O=GEANT Vereniging`) belongs to the GÉANT private CA hierarchy, whose root and intermediate certificates are not publicly distributed and could not be configured as trusted client CAs in the test server. This is a structural property of the IGTF authentication certificate model rather than a limitation of the token or the PKCS#11 integration: the card correctly enumerates and presents the certificate; the server-side trust configuration is simply not achievable without access to the private GÉANT CA bundle.

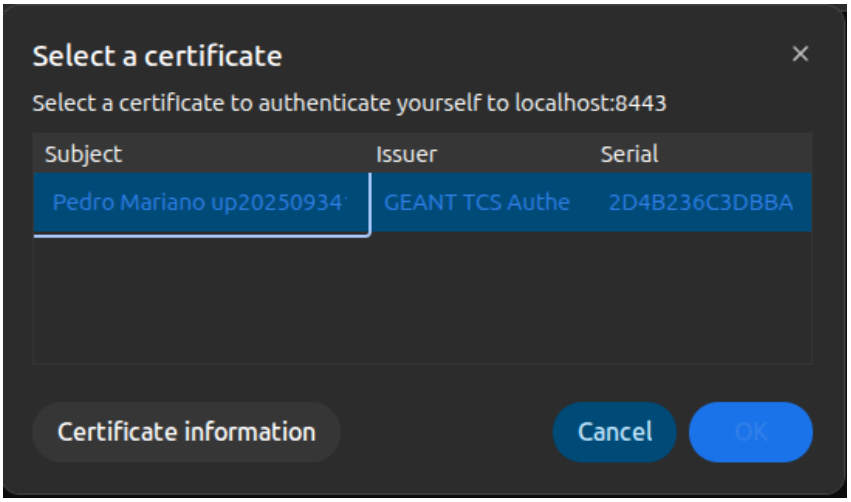

Figure 43: Chrome presenting the GÉANT TCS IGTF Personal certificate for TLS client authentication, confirming correct PKCS#11 enumeration of the institutional certificate stored on the hardware token.

Figure 44: Nginx returning a 400 SSL certificate error because the GÉANT TCS Authentication CA is part of a private trust hierarchy whose CA certificates are not publicly available for server-side trust configuration.

Table 9 summarizes the outcome of each scenario under the GÉANT TCS certificates.

Table 9: Scenario validation results with GÉANT TCS institutional certificates.

Scenario	**Result**	**Notes**
S/MIME signing	Pass	Signed and verified end-to-end via Thunderbird; issued by GEANT S/MIME RSA 1
SSH authentication	Pass	Hardware-backed login without password; IGTF Personal cert in slot 01
PAM authentication	Pass	Via `pam_p11` and shared `authorized_keys` mapping
TLS client auth	Partial	PKCS#11 enumeration correct; server-side trust impossible (private CA hierarchy)

6.11 Limitations and Real-World Deployment Considerations

The implemented prototype successfully demonstrates all target scenarios within the scope of an experimental evaluation. However, several components that are essential in a production deployment were deliberately omitted to keep the scope manageable. This section describes what would need to be added or changed to deploy an equivalent system in a real-world context.

Certificate revocation infrastructure: The prototype has no revocation mechanism. In a production system, the CA must publish Certificate Revocation Lists (CRLs) or operate an Online Certificate Status Protocol (OCSP) responder. Relying applications, including browsers, SSH clients and email clients, check revocation status before accepting a certificate. Without this infrastructure, a compromised or lost card cannot be reliably blocked, since the certificate remains cryptographically valid until its expiry date.

Hardware Security Module for the CA private key: The SAHC Root CA private key is stored in an XCA database on the host filesystem. In a production deployment, the CA private key must itself be protected inside a Hardware Security Module (HSM), a device offering equivalent tamper-resistance to a smartcard but designed for server-side use. An exposed CA private key allows an attacker to issue arbitrary trusted certificates, undermining the entire PKI.

Secure provisioning channel: Token provisioning in this prototype, meaning on-card key generation, certificate issuance and certificate storage, was performed directly at the operator's workstation. In production, provisioning must occur over an authenticated and encrypted channel, with the CA and the card reader under physical access control, to prevent certificate substitution or interception of the card during initialization.

PIN lifecycle management: The prototype uses a single static PIN with no policy controls. A real deployment requires PIN complexity requirements, mandatory change on first use, a defined number of retry attempts before card lockout, and a secure PIN reset or unlock mechanism that does not require physical destruction of the card.

Certificate lifecycle management: The certificates issued in this prototype have fixed validity periods and no automated renewal process. A production system requires tooling to monitor expiry dates, notify users ahead of time and support certificate renewal without requiring a full re-provisioning of the card.

Integration with enterprise identity infrastructure: The CN-to-username mapping used for PAM and SSH authentication is managed through a static configuration file. In an enterprise context, this mapping must be driven by a directory service such as LDAP or Active Directory, ensuring that access rights follow the organization's identity management processes and that deprovisioned users lose access immediately.

Audit logging: No audit trail of cryptographic operations is maintained in the prototype. Production PKI tokens in regulated environments must log every use of a private key, including the timestamp, the requesting application, and the result, to support forensic analysis and compliance reporting.

Physical security and card lifecycle: The prototype does not address what happens when a card is lost, stolen or damaged. A real deployment requires a process for issuing replacement cards, revoking the certificates of the original card, and ensuring that the replacement is provisioned through the same secure channel as the original. The card itself may also need to carry visible identity information (photograph, name) to allow physical verification of ownership.

7 Requirement Validation

This section presents the validation of each functional and security requirement defined in Section 4. Table 10 provides a summary, followed by detailed evidence for each item.

Table 10: Validation summary for functional and security requirements.

Requirement	Status	Evidence
FR1: Dual-certificate support	**Pass**	`pkcs15-tool --dump`
FR2: Hardware-backed operations	**Pass**	All scenarios
FR3: PKCS#11 exposure	**Pass**	`pkcs11-tool -L, -O`
FR4: PDF signing	**Pass**	`pdfsig`, LibreOffice
FR5: Auth and comm scenarios	**Pass**	SSH, TLS (Firefox + Chrome), PAM, S/MIME
FR6: Algorithm characterization	**Pass**	Benchmark table
FR7: Comparative evaluation	**Pass**	Section 6.8
SR1: Non-exportability	**Pass**	`neverExtract` flag (Figure 47), read failure (Figure 48)
SR2: On-card execution	**Pass**	All scenarios
SR3: Access control	**Pass**	PIN required in all scenarios
SR4: Certificate integrity	**Pass**	`certutil -V`, `pdfsig`
SR5: No host-side keys	**Pass**	Runtime uses token only
SR6: Interoperability	**Pass**	All applications unmodified
SR7: Limitations recorded	**Pass**	Section 6.6

FR1: Dual-certificate support: `pkcs15-tool --dump` confirms two certificates and two private key pairs on the token: ID 01 (Auth Certificate) and ID 02 (Sign Certificate), each individually accessible through OpenSC and with distinct Key Usage extensions. **Pass.**

FR2: Hardware-backed private-key operations: Private-key operations were performed across all scenarios (PDF signing, SSH, TLS, PAM, S/MIME signing and decryption) with the private key resident on the card. The `neverExtract` flag in the PKCS#15 metadata and the hardware-backed flag (`hw`) in the PKCS#11 mechanism list confirm on-card execution. **Pass.**

FR3: PKCS#11 exposure: `pkcs11-tool -O` enumerates all keys and certificates exposed through the PKCS#11 interface, as shown in Figure 45. The mechanism list confirms RSA hardware support. **Pass.**

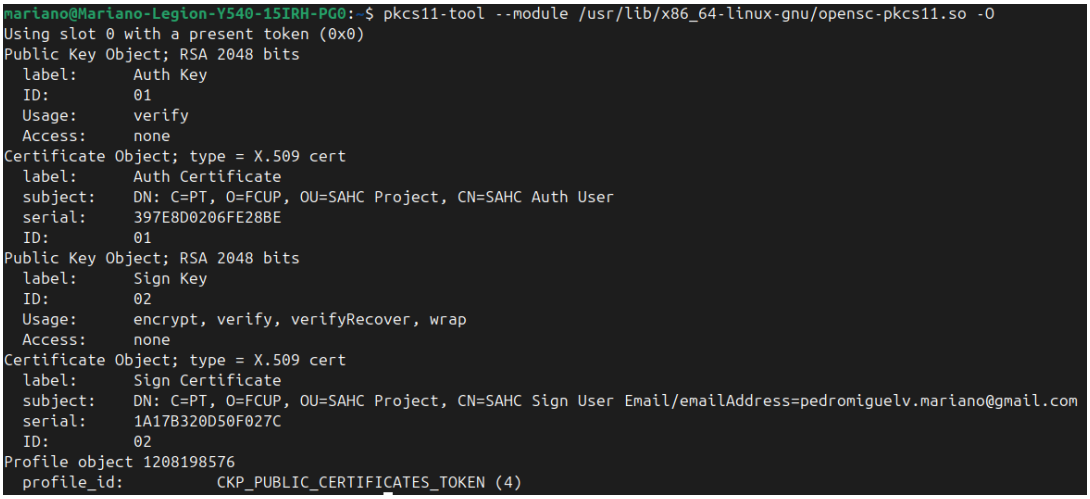

Figure 45: Output of `pkcs11-tool -O`: both public keys and X.509 certificates (Auth and Sign, IDs 01 and 02) are correctly enumerated through the OpenSC PKCS#11 interface.

FR4: PDF signing: Two independent methods were validated. The PDFBox-based `SmartcardSigner` produces a PDF verified as "Signature is Valid / Certificate is Trusted" by `pdfsig`. LibreOffice Draw produces a PAdES signature validated as "digitally signed and the signature is valid". **Pass.**

FR5: Authentication and communication scenarios: All four scenarios were successfully demonstrated with the private CA certificates:

- **SSH:** login via PKCS#11 token without password, confirmed by server greeting.
- **TLS**: mutual TLS with Nginx, confirmed by server response showing authenticated client DN, validated in both Firefox and Google Chrome.
- **PAM:** `pamtester` reports `successfully authenticated`.
- **S/MIME:** Thunderbird displays "Message Is Signed" and "Message Is Encrypted" with the correct certificate attribution.

The SSH, PAM and S/MIME scenarios were additionally validated with GÉANT TCS institutional certificates (Section 6.10), confirming that the PKCS#11 integration is certificate-agnostic and functions identically with both private CA and institutionally issued credentials. **Pass.**

FR6: Algorithm characterization: The supported mechanism list from `pkcs11-tool -M` shows exclusively RSA with `keySize={2048,2048}`, as shown in Figure 46. The `hw` flag on `RSA-PKCS` and `RSA-PKCS-KEY-PAIR-GEN` confirms hardware-accelerated execution. No elliptic curve mechanism is present. Attempts to generate RSA 1024, RSA 4096 and EC P-256/P-384 keys all failed with hardware-level errors. RSA 2048 signing averages 1.56 s per operation. See Table 3. **Pass.**

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so -M
Using slot 0 with a present token (0x0)
Supported mechanisms:
  SHA-1, digest
  SHA224, digest
  SHA256, digest
  SHA384, digest
  SHA512, digest
  MD5, digest
  RIPEMD160, digest
  GOSTR3411, digest
  RSA-PKCS, keySize={2048,2048}, hw, decrypt, sign, verify
  SHA1-RSA-PKCS, keySize={2048,2048}, sign, verify
  SHA224-RSA-PKCS, keySize={2048,2048}, sign, verify
  SHA256-RSA-PKCS, keySize={2048,2048}, sign, verify
  SHA384-RSA-PKCS, keySize={2048,2048}, sign, verify
  SHA512-RSA-PKCS, keySize={2048,2048}, sign, verify
  MD5-RSA-PKCS, keySize={2048,2048}, sign, verify
  RIPEMD160-RSA-PKCS, keySize={2048,2048}, sign, verify
  RSA-PKCS-KEY-PAIR-GEN, keySize={2048,2048}, hw, generate_key_pair
```

Figure 46: Output of `pkcs11-tool -M`: all RSA mechanisms are fixed at `keySize={2048,2048}` and hardware-accelerated (`hw`); no elliptic curve mechanism is present.

FR7: Comparative evaluation: A structural comparison with the Portuguese Citizen Card was performed using `pkcs15-tool --dump`, documenting differences in key size, PIN structure, usage flags, certificate chain storage and card mutability. Functional scenario testing was subsequently carried out with all three target scenarios: PDF signing succeeded using the Sign PIN and the qualified signature certificate (ID 46), producing a result verified as "Signature is Valid / Certificate is Trusted". SSH authentication succeeded after enabling the legacy `ssh-rsa` algorithm required by the CC's PKCS#11 driver. TLS mutual authentication was demonstrated via the OpenSSL PKCS#11 engine; Firefox-based and Chrome-based TLS both failed due to their prohibition of SHA-1 in TLS 1.2 handshake signatures. A full functional comparison table is provided in Section 6.8. PAM authentication succeeded using `pam_p11` with SHA-1 challenge signing. S/MIME signing succeeded via Thunderbird, with decryption confirmed as structurally impossible due to the `nonRepudiation` key usage. **Pass.**

SR1: Non-exportability: All private keys on the IsoApplet token carry `sensitive, alwaysSensitive, neverExtract, local` access flags in the PKCS#15 metadata, confirming that on-card generation permanently binds the key to the hardware. Figure 47 shows the complete token dump, with both private keys carrying the `neverExtract` flag and the two X.509 certificates correctly associated by ID. Furthermore, any attempt to read private key material directly from the token is rejected by the OpenSC middleware, as shown in Figure 48. **Pass.**

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pkcs15-tool --dump
Using reader with a card: bit4id miniLector-EVO 00 00
PKCS#15 Card [SAHC PKI Token]:
        Version        : 0
        Serial number  : 0000
        Manufacturer ID: unknown
        Last update    : 20260511105135Z
        Flags          : EID compliant


PIN [User PIN]
        Object Flags   : [0x03], private, modifiable
        ID             : 01
        Flags          : [0x39], case-sensitive, unblock-disabled, initialized, needs-padding
        Length         : min_len:4, max_len:16, stored_len:16
        Pad char       : 0x00
        Reference      : 1 (0x01)
        Type           : ascii-numeric

Private RSA Key [Auth Certificate]
        Object Flags   : [0x03], private, modifiable
        Usage          : [0x04], sign
        Access Flags   : [0x1D], sensitive, alwaysSensitive, neverExtract, local
        Algo_refs      : 0
        ModLength      : 2048
        Key ref        : 0 (0x00)
        Native         : yes
        Path           : 3f005015
        Auth ID        : 01
        ID             : 01
        MD:guid        : 9508e905-48b0-440a-4a61-e5743b76c1e3

Private RSA Key [Sign Certificate]
        Object Flags   : [0x03], private, modifiable
        Usage          : [0x2E], decrypt, sign, signRecover, unwrap
        Access Flags   : [0x1D], sensitive, alwaysSensitive, neverExtract, local
        Algo_refs      : 0
        ModLength      : 2048
        Key ref        : 1 (0x01)
        Native         : yes
        Path           : 3f005015
        Auth ID        : 01
        ID             : 02
        MD:guid        : 32a3cf4a-ef23-8d77-2788-31c3744c1327
```

```
Public RSA Key [Auth Key]
        Object Flags   : [0x02], modifiable
        Usage          : [0x40], verify
        Access Flags   : [0x00]
        ModLength      : 2048
        Key ref        : 0 (0x00)
        Native         : no
        Path           : 3f0050153300
        ID             : 01

Public RSA Key [Sign Key]
        Object Flags   : [0x02], modifiable
        Usage          : [0xD1], encrypt, wrap, verify, verifyRecover
        Access Flags   : [0x00]
        ModLength      : 2048
        Key ref        : 0 (0x00)
        Native         : no
        Path           : 3f0050153301
        ID             : 02

X.509 Certificate [Auth Certificate]
        Object Flags   : [0x02], modifiable
        Authority      : no
        Path           : 3f0050153400
        ID             : 01
        Encoded serial : 02 08 397E8D0206FE28BE

X.509 Certificate [Sign Certificate]
        Object Flags   : [0x02], modifiable
        Authority      : no
        Path           : 3f0050153401
        ID             : 02
        Encoded serial : 02 08 1A17B320D50F027C
```

Figure 47: PKCS#15 token dump: PIN policy, private RSA keys with `neverExtract` access flags (left), and public keys with X.509 certificates for authentication (ID 01) and signing (ID 02) (right).

```
mariano@Mariano-Legion-Y540-15IRH-PG0:~$ pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
    --login --pin 1234 --id 01 --type privkey --read-object
pkcs11-tool --module /usr/lib/x86_64-linux-gnu/opensc-pkcs11.so \
    --login --pin 1234 --id 02 --type privkey --read-object
Using slot 0 with a present token (0x0)
sorry, reading private keys not (yet) supported
Using slot 0 with a present token (0x0)
sorry, reading private keys not (yet) supported
```

Figure 48: Attempted export of both private keys via `pkcs11-tool`: the operation is refused by the middleware, confirming that private key material cannot be extracted from the token.

SR2: On-card execution: All cryptographic operations were invoked through the PKCS#11 interface and delegated to the card. No private key material was present on the host at runtime. The `hw` flag in the mechanism list confirms hardware-accelerated execution inside the token. **Pass.**

SR3: Access control: Every private-key operation triggers a PIN prompt. In all tested scenarios (PDF, SSH, TLS, PAM, S/MIME), the PIN was required before the operation could proceed, confirming that access control is enforced by the token. **Pass.**

SR4: Certificate integrity: `certutil -V -u S` on the signing certificate returns `certificate is valid`, confirming a correct chain from the end-entity certificate to the SAHC Root CA. PDF signatures verified by `pdfsig` further confirm end-to-end integrity. **Pass.**

SR5: No host-side keys: After the initial provisioning phase, no software key files are required or used for any operational scenario. All runtime operations access private

keys exclusively through the PKCS#11 interface to the hardware token. **Pass.**

SR6: Interoperability: All five application scenarios were executed using unmodified third-party software (OpenSSH, Firefox, Nginx, libpam-pkcs11, Thunderbird, LibreOffice) through the standard OpenSC PKCS#11 interface. No modifications to application source code were required. **Pass.**

SR7: Limitations recorded: The following limitations were observed and documented:

- RSA 1024 and RSA 4096 are not supported by the card hardware.
- EC key generation is not supported (`CKM_EC_KEY_PAIR_GEN` absent from mechanism list).
- PKCS#15 key usage flags must include `decrypt` for S/MIME decryption to function; sign-only keys cause `CKR_KEY_TYPE_INCONSISTENT` in `C_Decrypt`.
- Thunderbird NSS certificate management requires the signing certificate to be absent from the software NSS database to ensure the PKCS#11 token path is used for decryption.
- The Portuguese Citizen Card's PKCS#11 driver in OpenSC 0.25.0 only accepts `CKM_RSA_PKCS` with SHA-1 DigestInfo for the authentication key; SHA-256 DigestInfo causes `CKR_ARGUMENTS_BAD`. This requires enabling the deprecated `ssh-rsa` algorithm for SSH and forces TLS to use SHA-1 client certificate signatures, which modern browsers such as Firefox refuse by internal security policy, making browser-based TLS mutual authentication impossible without the official AMA middleware.
- The CC's signature key carries `nonRepudiation` usage only, making S/MIME decryption structurally impossible via OpenSC.
- The official AMA middleware (`pteid-mw`), which would likely resolve the SHA-256 incompatibility, is not available as a native package for Ubuntu 24.04.

8 Conclusion

This project evaluated the IsoApplet as a basis for a practical, open-source PKI token using a programmable JavaCard. A complete system was designed, provisioned and validated, covering all five target application scenarios: PDF digital signature, SSH authentication, TLS client authentication, Linux PAM authentication and S/MIME email signing and encryption.

The prototype demonstrates that the combination of IsoApplet, OpenSC and PKCS#11 is sufficient for a realistic multi-purpose PKI token. All functional and security requirements were met, with the private keys remaining on-card throughout all operations, PIN-based access control enforced in every scenario and unmodified third-party applications consuming the token transparently. The card was additionally provisioned with institutional certificates issued through the GÉANT TCS (HARICA Generation 5), and the SSH, PAM and S/MIME scenarios were re-validated with these credentials, confirming that the PKCS#11 integration is certificate-agnostic and transfers directly to

institutionally issued credentials without any change to the middleware or application configuration.

However, several limitations inherent to the selected hardware platform were identified. The card supports only RSA 2048, with no support for RSA 1024, RSA 4096 or any elliptic curve mechanism. The signing performance of approximately 1.56 s per RSA 2048 operation is adequate for interactive use but would be a constraint in high-throughput environments. The PKCS#15 key usage configuration has a direct and non-obvious effect on PKCS#11 behavior, requiring explicit `sign,decrypt` usage for S/MIME decryption to function correctly.

The comparison with the Portuguese Citizen Card highlights the trade-offs between a commodity open-source token and a nationally certified credential. The Citizen Card uses larger RSA keys (3072 bits), enforces finer-grained PIN separation for qualified signatures and embeds the full certificate chain on the card itself. Functional testing demonstrated that PDF signing, SSH and TLS client authentication are achievable with the Citizen Card via OpenSC, though with an important caveat: the CC's PKCS#11 driver only supports `CKM_RSA_PKCS` with SHA-1 DigestInfo, causing incompatibilities with modern software stacks that default to SHA-256. This required enabling the deprecated `ssh-rsa` algorithm for SSH and precluded browser-based TLS client authentication entirely. The IsoApplet, despite its simpler hardware, exhibits no such restriction and interoperates transparently with all tested applications. These findings suggest that interoperability with modern cryptographic protocols is not solely a function of key size or certificate quality, but also of the completeness of the PKCS#11 driver implementation and the hash algorithm preferences of each application layer.

Overall, the IsoApplet-based token proves to be a viable and interoperable PKI solution for experimental and educational purposes. Its open-source nature makes it transparent and auditable, and its compatibility with standard middleware ensures that real applications can use it without modification. The Citizen Card, by contrast, illustrates the additional engineering investment required to deploy a nationally trusted identity document: a dedicated middleware ecosystem, a state PKI hierarchy with embedded certificate chains, fine-grained PIN separation per operation class and a design optimised for long-term deployment in a regulated environment. These are properties that cannot be replicated by a generic programmable card, regardless of the applet installed on it.

References

[1] D. Cooper et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. IETF, 2008.

[2] Philip Wendland. *IsoApplet: A JavaCard PKI Applet*. `https://github.com/philipWendland/IsoApplet`. Accessed: March 2026. 2023.

[3] OpenSC Project. *OpenSC – Open Source Smart Card Tools and Libraries*. `https://github.com/OpenSC/OpenSC`. Accessed: March 2026. 2024.

[4] OASIS PKCS 11 TC. *PKCS #11 Cryptographic Token Interface Standard, Version 3.1*. Tech. rep. OASIS, 2023.

[5] Agência para a Modernização Administrativa (AMA). *Cartão de Cidadão – Especificações Técnicas*. `https://www.autenticacao.gov.pt`. Accessed: March 2026. 2024.

[6] Oracle Corporation. *Java Card 2.2.2 Platform Specification*. https://www.oracle.com/java/technologies/javacard-sdk-downloads.html. 2006.

[7] RSA Security. *PKCS #15: Cryptographic Token Information Format Standard, Version 1.1*. Tech. rep. 2000.

[8] pcsc-lite project. *pcsc-lite – Middleware to Access a Smart Card Using SCard API (PC/SC)*. https://pcsclite.apdu.fr/. Accessed: March 2026. 2026.

[9] Martin Paljak. *GlobalPlatformPro – The Swiss Army Knife for JavaCards*. https://github.com/martinpaljak/GlobalPlatformPro. Accessed: March 2026. 2026.

[10] Christian Hohnstaedt. *XCA – X Certificate and Key Management*. https://www.hohnstaedt.de/. Accessed: March 2026. 2026.

[11] The Apache Software Foundation. *Apache PDFBox – A Java PDF Library*. https://pdfbox.apache.org. Accessed: March 2026. 2024.

[12] OpenBSD Project. *OpenSSH – Keeping Your Communication Secret*. https://www.openssh.com. Accessed: March 2026. 2024.

[13] B. Ramsdell and S. Turner. *S/MIME Version 3.2 Message Specification*. RFC 5751. IETF, 2010.

[14] Mozilla Support. *Instructions for obtaining a personal S/MIME certificate by creating a CSR*. https://support.mozilla.org/en-US/kb/instructions-smime-certificate-using-csr. Accessed: March 2026. 2025.